MACHINE LEARNING CURRICULUM — MODULE 1

INTRODUCTION TO MACHINE LEARNING

From Absolute Beginner to Strong Foundational Understanding

A Comprehensive University-Level Textbook and Professional Training Handbook

Kajola Gbenga Data Scientist & AI Engineer

Table of Contents

1. What Is Artificial Intelligence?
2. What Is Machine Learning?
3. Traditional Programming vs Machine Learning
4. Why Machine Learning?
5. How Machine Learning Works
6. Core Machine Learning Terminology
7. Features
8. Target / Label
9. Types of Machine Learning
10. Supervised Learning
11. Unsupervised Learning
12. Semi-Supervised Learning
13. Self-Supervised Learning
14. Reinforcement Learning
15. Batch vs Online Learning
16. Instance-Based vs Model-Based Learning
17. Parametric vs Non-Parametric Models
18. Training, Validation and Testing
19. What Does “Learning” Mean Mathematically?
20. Generalization
21. Bias and Variance
22. ML vs Data Analytics vs Data Science
23. Machine Learning vs Deep Learning
24. Machine Learning vs Generative AI
25. Real-World Machine Learning Project
26. Your First Machine Learning Model
27. Machine Learning Project Structure
28. Common Beginner Mistakes
29. Machine Learning Business Use Cases
30. Machine Learning Decision Framework
31. Machine Learning Tool Ecosystem
32. Machine Learning Career Map
33. Practical Exercises
34. Mini Case Studies
35. Interview Preparation
36. Coding Challenges
37. Troubleshooting
38. Professional Machine Learning Workflow
39. Baseline Models
40. ML Ethics and Responsible Machine Learning

41. Final Comprehensive Project
42. Final Knowledge Check
43. Module Summary
44. Master Cheat Sheet

1. What Is Artificial Intelligence?

1.1 Simple Definition

Artificial Intelligence (AI) is the field of building computer systems that perform tasks which, when performed by humans, we would describe as requiring intelligence — recognizing a face, understanding a sentence, making a decision, planning a route, learning from experience.

Technical definition: AI is the study and construction of *rational agents* — systems that perceive an environment through sensors or data, reason about that perception, and act to maximize their chance of achieving a defined objective.

Intuition: A calculator performs arithmetic faster than any human, but we don't call it intelligent — because the rules were entirely written by a person, and the calculator never deviates, adapts, or handles anything outside those rules. AI begins where a system must handle situations its designers did not explicitly enumerate.

1.2 Why Artificial Intelligence Was Developed

AI emerged from a practical observation: an enormous class of valuable problems cannot be solved by writing explicit instructions, because nobody can *state* the instructions. You can describe how to compute tax owed; you cannot describe, rule by rule, how to recognize your mother's face in a photograph, or how to tell a sarcastic sentence from a sincere one. AI exists to attack that second class of problem.

1.3 What Problems AI Attempts to Solve

Problem Class	Example	Why Rules Fail
Perception	Recognizing a tumor in an X-ray	The visual signature varies enormously; no finite rule list captures it
Language understanding	Determining whether a customer review is positive	Meaning depends on context, idiom, sarcasm, negation
Prediction under uncertainty	Will this borrower default?	The answer depends on dozens of interacting factors with no deterministic formula
Sequential decision-making	Playing chess, routing a delivery fleet	The number of possible futures is astronomically large
Personalization at scale	Which product should this specific user see?	The "rule" would need to be different for every one of 50 million users

1.4 Traditional Software Systems vs AI Systems

```
TRADITIONAL SOFTWARE
    Human writes rules  →  Program applies rules  →  Output
```

```
    The programmer is the source of all knowledge in the system.

AI SYSTEM
    Human supplies data/objective  →  System derives behavior  →
Output
    The data (or the environment) is the source of much of the
knowledge.
```

Why this matters professionally: in traditional software, a wrong output means a bug in the code. In an AI system, a wrong output may mean bad data, an inappropriate objective, a distribution shift, or a genuinely irreducible uncertainty — a fundamentally different debugging discipline (Part 37).

1.5 Rule-Based Systems

A **rule-based system** encodes knowledge as explicit `IF...THEN` rules.

```
IF transaction_amount > 10000 AND country != home_country
THEN flag_for_review
```

Where they shine: regulated, auditable domains where the rule must be stated, defended, and unchanged — tax computation, payroll, compliance thresholds. **Where they break:** when the number of rules needed explodes, when rules start contradicting each other, and when the underlying pattern shifts faster than humans can rewrite rules. A mature fraud rules-engine in a bank may hold thousands of rules that no single person fully understands — which is precisely the pain that motivates moving to ML (Part 3).

1.6 Expert Systems

An **expert system** is a rule-based system that attempts to capture a human domain expert's decision-making, typically combining a *knowledge base* (the rules/facts) with an *inference engine* (which chains rules to reach conclusions). MYCIN (1970s, diagnosing bacterial infections) is the classic example. They were the commercial face of AI in the 1980s and largely stalled for two reasons: **knowledge acquisition** (extracting rules from experts is slow and experts often cannot articulate what they know) and **brittleness** (performance collapses just outside the encoded knowledge).

1.7 Knowledge-Based Systems

A broader family in which the system's knowledge is stored **explicitly and separately** from the reasoning procedure — ontologies, knowledge graphs, semantic networks. Modern relevance is high: enterprise **knowledge graphs** are widely used today, and they increasingly pair with LLMs in retrieval-augmented systems (Part 24) to supply verifiable facts that a language model alone cannot guarantee.

1.8 Search and Optimization in AI

A large part of classical AI is **search**: representing a problem as states and transitions, then finding a path to a goal state. Route planning, game playing, scheduling, and resource allocation are all search problems. **Optimization** — finding the input that minimizes or maximizes an

objective — is the sibling concept, and is the mathematical machinery that ML itself ultimately runs on (Part 19): training a model *is* an optimization problem.

1.9 Machine Learning as a Branch of AI

Machine Learning is the subfield of AI in which the system's behavior is *derived from data* rather than explicitly programmed. It is not the whole of AI (search, planning, knowledge representation and symbolic reasoning are AI but not ML), but it is the subfield responsible for essentially all of AI's commercial impact in the last two decades.

1.10 Deep Learning as a Branch of Machine Learning

Deep Learning is the subfield of ML using *neural networks with many layers*, which learn their own internal representations of the data rather than relying on human-engineered features. It is the reason computer vision and NLP advanced so dramatically after roughly 2012. Covered comparatively in Part 23 and deeply in a later module.

1.11 Generative AI

Generative AI describes models that *produce new content* — text, images, audio, code — rather than producing a label or a number. Technically most are deep learning models trained with self-supervised objectives (Part 13). Contrasted against predictive ML in Part 24.

1.12 Large Language Models

LLMs are very large neural networks (typically Transformer-based) trained on enormous text corpora to predict text, acquiring in the process a broad ability to summarize, translate, answer questions, write code, and follow instructions. They are simultaneously: deep learning (architecture), self-supervised learning (training objective), and generative AI (output type) — a useful illustration that these categories describe *different axes*, not a single hierarchy.

1.13 Computer Vision

The subfield concerned with extracting meaning from images and video — classification, object detection, segmentation, OCR, face recognition. Industrial examples: defect detection on a manufacturing line, pipeline corrosion inspection from drone imagery in oil & gas, radiology triage in healthcare.

1.14 Natural Language Processing

The subfield concerned with human language — sentiment analysis, named entity recognition, translation, summarization, question answering. Industrial examples: AML (anti-money-laundering) adverse-media screening in banking, support-ticket routing in telecom, clinical-note extraction in healthcare.

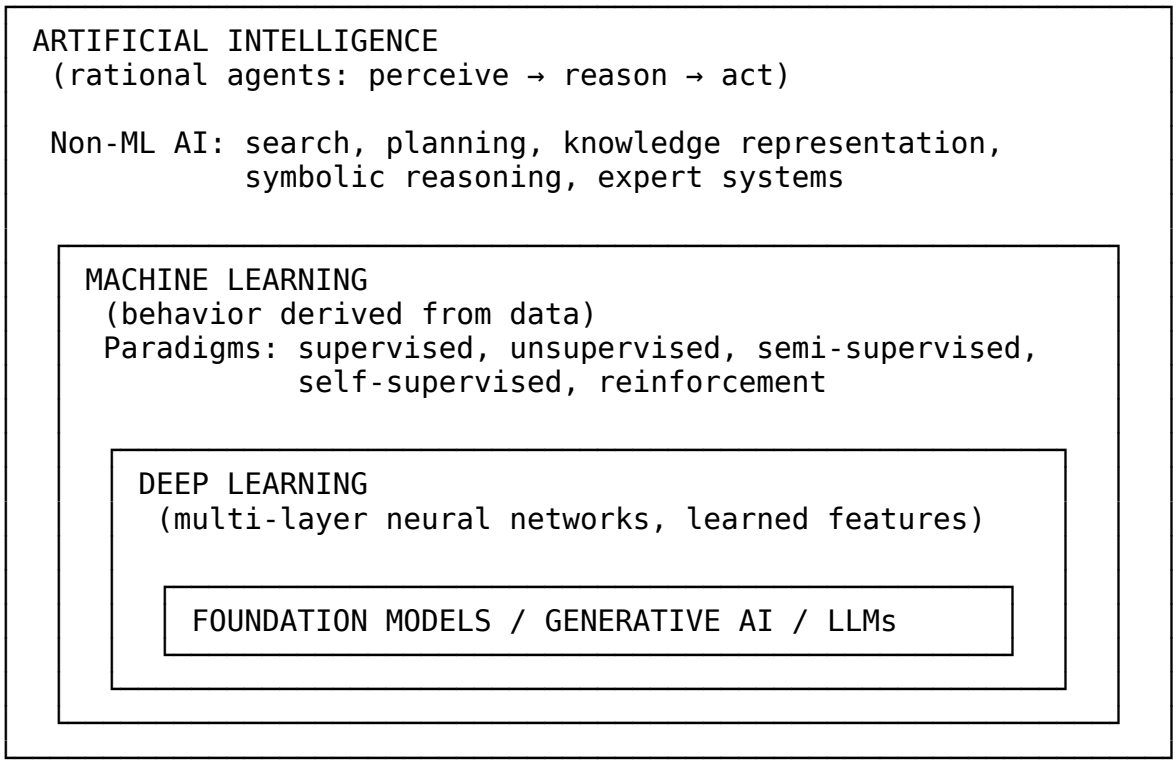
1.15 Robotics

The engineering discipline of physically embodied agents. AI contributes perception (vision), planning (search), and control (often reinforcement learning). Distinguished from the others by having to act in a physical, noisy, safety-critical world in real time.

1.16 Reinforcement Learning

A *learning paradigm* (not an application domain) in which an agent learns by acting in an environment and receiving rewards. Introduced properly in Part 14.

1.17 How Everything Fits Together




```
APPLICATION DOMAINS cut ACROSS all of the above:
  Computer Vision · NLP · Speech · Robotics · Recommender Systems
  (each can be solved with classical AI, classical ML, or deep
learning)
```

The critical insight for a beginner: *Computer Vision* and *NLP* are **problem domains**; *supervised/unsupervised/reinforcement* are **learning paradigms**; *deep learning* is a **model family**; *generative AI* is an **output type**. A single system — say, an LLM-powered document extractor — is simultaneously in all four categories. Beginners frequently mis-model these as one nested hierarchy; they are orthogonal axes.

2. What Is Machine Learning?

2.1 Several Definitions

Plain language: Machine Learning is how we get a computer to figure out the rules by itself, by showing it examples, instead of us writing the rules by hand.

Arthur Samuel (1959), the classic: "The field of study that gives computers the ability to learn without being explicitly programmed."

Tom Mitchell (1997), the precise one: "A computer program is said to learn from experience **E** with respect to some class of tasks **T** and performance measure **P**, if its performance at tasks in T, as measured by P, improves with experience E."

Mitchell's definition is worth memorizing because it forces you to name three things that beginners routinely leave vague. For spam detection: **T** = classify an email as spam or not; **E** = a labelled archive of past emails; **P** = the percentage of emails correctly classified (or better, the false-positive rate on legitimate mail). If you cannot state T, E and P for your project, you do not yet have an ML problem — you have a wish.

2.2 Machine Learning From Seven Perspectives

Perspective	Machine Learning is…
Simple	Learning rules from examples instead of being told the rules
Technical	Fitting a parameterized function to data such that it generalizes to unseen data from the same distribution
Mathematical	Optimization: find parameters θ minimizing a loss function L over the data — $\theta^* = \text{argmin } L(\theta)$ (Part 19)
Software engineering	A system whose logic is *compiled from data* rather than written, requiring data pipelines, versioning, testing, deployment and monitoring (Parts 27, 38)
Business	A way to convert historical data into repeatable, automated decisions at a scale and speed humans cannot match
Data Scientist's	The disciplined process of framing a business question as a learnable task, engineering the data that makes it learnable, and quantifying how much to trust the answer
ML Engineer's	A production system with unusual failure modes — data drift, training-serving skew, silent degradation — requiring monitoring and retraining, not just deployment

2.3 The Core Idea

```
   DATA  →  PATTERNS  →  MODEL  →  PREDICTION / DECISION
    │           │           │              │
    │           │           │              └── acted on by a business
process
    │           │           └── a compact, reusable encoding of the
patterns
    │           └── statistical regularities that hold across many
examples
    └── historical examples where we know what happened
```

2.4 What Does It Mean for a Machine to "Learn"?

It does **not** mean the machine understands anything. "Learning" here has a precise, modest meaning: **the system adjusts internal numbers (parameters) so that its outputs on data it has never seen get better according to a measurable criterion.**

A useful analogy: imagine tuning an old radio. You turn the dial (the parameter) and listen to the static (the loss). You keep turning in whatever direction reduces static. Eventually you land on a clear station. The radio did not "understand" broadcasting — it was just tuned, by a process that used feedback. Machine learning is that, with millions of dials and an automated procedure for deciding which way to turn each one.

Critically: learning requires the future to resemble the past. A model learns patterns from historical data and assumes those patterns still hold when it makes predictions. When that assumption breaks — a pandemic, a regulatory change, a new fraud technique — the model degrades even though its code is unchanged. This is why monitoring (Part 5, stage 16) is not optional.

2.5 Ten Worked Examples

For each: **Input data → Features → Target → Pattern → Model → Prediction → Business decision.**

1. Email spam detection. Input: historical emails marked spam/not-spam. Features: word frequencies, sender reputation, presence of links, time sent. Target: `is_spam` (binary). Pattern: certain word combinations and sender behaviours co-occur with spam. Model: a classifier. Prediction: probability this new email is spam. Decision: route to inbox, spam folder, or quarantine.

2. House price prediction. Input: past property sales. Features: square footage, bedrooms, location, age, condition, distance to schools. Target: `sale_price` (numeric). Pattern: price rises with size and desirable location, non-linearly. Model: a regressor. Prediction: expected price of a new listing. Decision: set an asking price; flag under/overpriced listings.

3. Customer churn prediction (telecom). Input: 24 months of subscriber records. Features: tenure, monthly spend, support-call count, contract type, data usage trend. Target: `churned_within_30_days` (binary). Pattern: rising support calls plus a month-to-month contract plus declining usage precede cancellation. Model: a classifier. Prediction: churn

probability per subscriber. Decision: route the top 5% risk into a retention campaign — and note the ROI depends on *ranking quality*, not raw accuracy (Part 28).

4. Fraud detection (banking). Input: historical transactions with confirmed fraud labels. Features: amount, merchant category, time since last transaction, geographic distance from previous transaction, device fingerprint. Target: `is_fraud` (binary, heavily imbalanced — often <0.5%). Pattern: unusual velocity and geography combinations. Model: a classifier, often with anomaly-detection support. Prediction: fraud score. Decision: approve, challenge with step-up authentication, or block.

5. Credit risk prediction. Input: past loan applications and their outcomes. Features: income, debt-to-income ratio, credit history length, prior delinquencies, employment tenure. Target: `defaulted` (binary). Pattern: leverage and repayment history dominate. Model: often a regularized logistic regression or gradient-boosted trees — *chosen partly because regulators require explainability* (Part 40). Prediction: probability of default. Decision: approve/decline, set interest rate and credit limit.

6. Product recommendation (e-commerce). Input: purchase and browsing history. Features: user-item interaction matrix, item attributes, session context. Target: often implicit (did the user click/buy?). Pattern: users with similar histories like similar items. Model: collaborative filtering or a ranking model. Prediction: ranked list of items. Decision: what to display on the homepage.

7. Customer segmentation (marketing). Input: customer attributes and behaviour. Features: recency, frequency, monetary value, category preferences. Target: **none** — this is unsupervised (Part 11). Pattern: customers cluster into natural groups. Model: a clustering algorithm. Output: segment assignments. Decision: tailor campaigns per segment.

8. Disease-risk prediction (healthcare). Input: electronic health records. Features: age, vitals, lab results, comorbidities, medication history. Target: `developed_condition_within_12_months`. Pattern: combinations of lab values and history elevate risk. Model: a classifier. Prediction: risk score. Decision: enrol high-risk patients into a preventive care programme.

9. Demand forecasting (retail/supply chain). Input: historical sales by SKU and store. Features: past sales, seasonality, promotions, price, holidays, weather. Target: `units_sold_next_week` (numeric). Pattern: strong seasonality plus promotion lift. Model: a time-series or regression model — **and the train/test split must be time-based, never random** (Part 18). Prediction: expected units. Decision: purchase orders and inventory allocation.

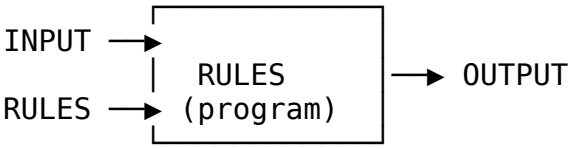
10. Predictive maintenance (oil & gas / manufacturing). Input: sensor telemetry from equipment plus maintenance records. Features: vibration, temperature, pressure, operating hours, rolling statistics of each. Target: `failure_within_7_days` (binary). Pattern: characteristic sensor drift precedes failure. Model: a classifier on engineered time-window features. Prediction: failure probability. Decision: schedule maintenance before failure, avoiding unplanned downtime costing far more than the intervention.

Notice what these share: **the model is never the deliverable — the business decision is.** A churn model nobody acts on has zero value regardless of its accuracy.

---

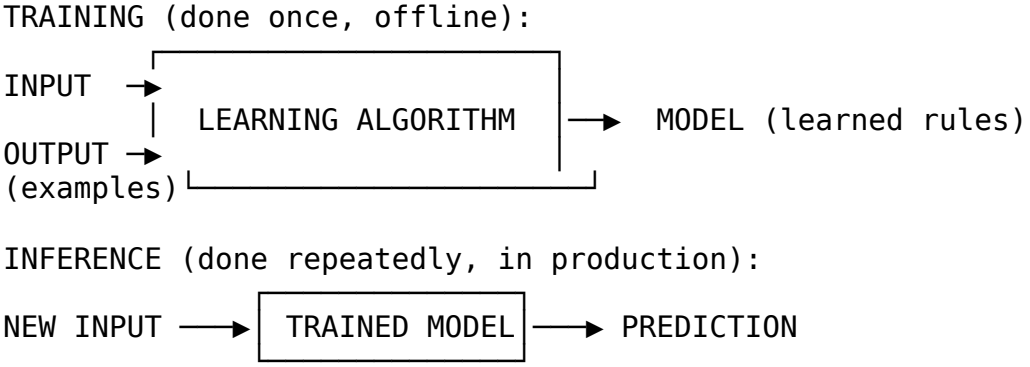
3. Traditional Programming vs Machine Learning

3.1 Traditional Programming



The programmer knows the rules and writes them. The computer executes them.

3.2 Machine Learning



The programmer supplies *examples of the right answer* and a learning procedure. The algorithm produces the rules.

The inversion is the whole point: in traditional programming, rules are the input and answers are the output. In ML, answers are the input (during training) and rules are the output.

3.3 Side-by-Side Examples

Task	Approach	Why
Tax calculation	Traditional	The rules are published law. They must be applied exactly, be auditable, and produce identical results for identical inputs. ML here would be strictly worse: less accurate, unexplainable, and legally indefensible.
Fraud detection	Machine Learning	Nobody can write the rules — fraudsters change tactics continuously, and the signal

Task	Approach	Why
		is a subtle combination of dozens of variables.
Payroll processing	Traditional	Deterministic, specified, regulated.
Handwriting recognition	Machine Learning	Try writing the rules for "this squiggle is a 7." You cannot.
Interest accrual	Traditional	A formula exists.
Credit limit optimization	Machine Learning	The optimal limit depends on predicted default risk and predicted spend — both learned.
Inventory reorder point at a fixed threshold	Traditional	If the business rule is "reorder at 100 units," implement it.
Demand forecast driving that reorder point	Machine Learning	The forecast itself is a pattern-recognition problem.

A professional pattern worth internalizing: real systems are usually *hybrid*. A production fraud system typically runs hard-coded regulatory rules (block sanctioned countries — non-negotiable, rule-based) **alongside** an ML risk score (learned). Neither replaces the other.

3.4 Why Some Problems Resist Hand-Written Rules

1. **Combinatorial explosion.** Fraud depends on ~50 variables interacting. Enumerating rules across their combinations is not humanly feasible.
2. **Tacit knowledge.** Experts recognize patterns they cannot verbalize. A radiologist "just sees" an anomaly.
3. **Non-stationarity.** The pattern changes faster than rules can be rewritten.
4. **Individual variation.** A rule that works on average is wrong for most specific individuals — personalization needs a function, not a rule.
5. **Perceptual input.** Pixels and audio waveforms have no natural rule-friendly representation.

3.5 Explicit Rules, Learned Rules, Pattern Recognition, Generalization

Explicit rules are written by humans, inspectable, and exactly as good as the human's knowledge. **Learned rules** are derived from data, often not human-readable, and exactly as good as the data. **Pattern recognition** is the act of detecting regularity in data. **Generalization** (Part 20) is the property that matters: performing well on data never seen during training. A system that perfectly reproduces its training data but fails on new data has *memorized*, not *learned* — the single most important distinction in this entire module.

---

4. Why Machine Learning?

4.1 Problem Classes Where ML Earns Its Place

Class	Question It Answers	Example
Prediction	What will happen?	Next month's demand
Classification	What kind of thing is this?	Fraud / legitimate
Ranking	What order should these go in?	Search results, lead priority
Recommendation	What should this person see next?	Product feed
Anomaly detection	What is unusual here?	Network intrusion, equipment fault
Forecasting	What will this time series do?	Energy load, cash flow
Optimization	What is the best action given predictions?	Dynamic pricing, route planning
Personalization	How should this differ per user?	Tailored content, individualized pricing
Automation	Can we remove the human from a repetitive judgment?	Document triage, first-pass claims review

Common preconditions: **large datasets** (patterns need enough examples to be distinguishable from noise) and **complex patterns** (if a simple rule works, use the simple rule).

4.2 When Traditional Programming Is the Better Choice

Not every problem requires Machine Learning. Prefer rules when:

- The rule is **known and stable** (tax, interest, statutory compliance).
- The decision must be **exactly reproducible and auditable**.
- You have **very little data** — a model trained on 200 examples of a rare event is usually worse than a domain expert's heuristic.
- **Errors are catastrophic and unacceptable**, and a rule provides a guarantee that a probabilistic model cannot.
- The **cost of building and maintaining** the ML system exceeds the incremental benefit over a heuristic.

A frequent and expensive failure mode in industry is a team building an ML system for a problem that a well-chosen `CASE WHEN` statement solves at 95% of the quality for 2% of the cost.

4.3 The Nine Decision Factors

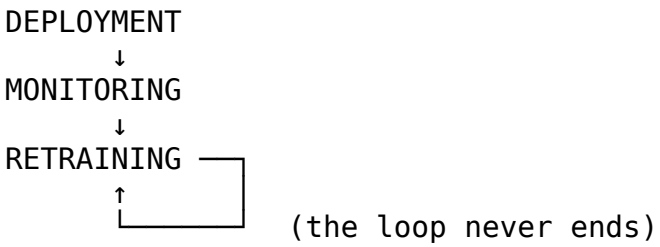
Before committing to an ML approach, evaluate each of these honestly:

Factor	Question to Answer
Business value	What decision changes because of this prediction, and what is that worth in money?
Cost	Build cost, compute cost, and the ongoing cost of maintaining it forever
Complexity	Do we have the skills and infrastructure to run this in production?
Data availability	Do we have enough labelled historical examples — and will we keep getting them?
Accuracy requirements	What accuracy makes this useful? If 99.9% is required and we can reach 85%, stop now.
Explainability	Must we explain individual decisions? (Credit, hiring, healthcare, insurance — usually yes, often by law.)
Latency	Milliseconds (real-time fraud) or overnight (monthly churn scoring)? This determines the entire architecture.
Maintenance	Who retrains it? Who monitors it? What happens when they leave?
Regulatory requirements	GDPR, fair-lending rules, medical device regulation, model risk management (SR 11-7 in US banking)

5. How Machine Learning Works

5.1 The Complete Lifecycle

```
BUSINESS PROBLEM
      ↓
DATA COLLECTION
      ↓
DATA UNDERSTANDING
      ↓
DATA CLEANING
      ↓
EDA (Exploratory Data Analysis)
      ↓
FEATURE ENGINEERING
      ↓
MODEL TRAINING
      ↓
VALIDATION
      ↓
EVALUATION
      ↓
```



5.2 The Seventeen Stages in Depth

1. Define the problem. Translate a business need into a learnable task with a measurable objective. "Reduce churn" is not a task. "Predict which subscribers will cancel within 30 days, so the retention team can contact the top 5% by risk" is. Specify Mitchell's T, E and P (Section 2.1). **Most failed ML projects fail here**, not in modelling.

2. Collect data. Identify sources (transactional databases, event streams, third-party data), obtain access, and confirm you can get this data *again* at prediction time. A feature available in your historical warehouse but not available at the moment of a live decision is unusable — a surprisingly common and painful discovery.

3. Understand the data. Learn what each column means from the people who generate it. `status = 3` means something specific to the billing team. Misunderstanding a column's semantics silently invalidates everything downstream.

4. Clean the data. Handle missing values, duplicates, impossible values (negative ages, future dates), inconsistent encodings ("NG", "Nigeria", "nigeria"), and outliers. Expect this and the next three stages to consume the majority of project time.

5. Explore the data (EDA). Compute distributions, correlations, class balance, and target relationships. Plot things. EDA is where you discover that 40% of your target column is missing, or that your "fraud" class is 0.2% of rows, or that a feature perfectly predicts the target because it is *derived from* the target (leakage — Part 18).

6. Prepare features. Encode categoricals, scale numerics where required, create derived features (ratios, time-window aggregates, differences). Covered in Part 7 and deeply in the feature-engineering module.

7. Define target. Make the label precise. "Churn" must mean something exact: no activity for 30 days? An explicit cancellation event? Non-renewal at contract end? Different definitions produce different models and different business meanings.

8. Split data. Into training, validation and test sets (Part 18) — using the *correct* splitting strategy for the data type.

9. Select algorithm. Choose candidates appropriate to the problem type, data size, interpretability requirements and latency budget. Start simple.

10. Train model. Run the learning algorithm on the training set to fit parameters (Part 19).

11. Validate model. Use the validation set (or cross-validation) to compare candidate models and tune hyperparameters — *without touching the test set*.

12. Evaluate model. Measure final performance on the untouched test set, using metrics that reflect the business objective (Part 28 — accuracy is usually the wrong metric).

13. Tune model. Adjust hyperparameters, features, or the algorithm, re-validating each time.

14. Interpret model. Understand *why* it predicts what it predicts — feature importances, partial dependence, individual explanations. Required for trust, for regulatory compliance, and for catching leakage.

15. Deploy model. Make it available to the system that will act on it: a batch job writing scores to a table, or a real-time API. This is a software engineering task (Part 27).

16. Monitor model. Track input distributions, prediction distributions, and — as labels arrive — actual performance. Models degrade silently; without monitoring you find out from a business stakeholder months later.

17. Retrain model. On a schedule or triggered by drift, retrain on fresher data and validate that the new model is genuinely better before promoting it.

5.3 Machine Learning Is Not `fit()` and `predict()`

```
# This is roughly 5% of a real ML project:
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

The other 95% is problem framing, data acquisition, cleaning, feature engineering, validation design, evaluation against business metrics, deployment, and monitoring. A beginner who learns only the two lines above can produce a number; a professional produces a *system that continues to create value*. This module exists to build that distinction from the start.

6. Core Machine Learning Terminology

Reference glossary. Examples use a customer-churn dataset throughout.

Term	Meaning	Example
Dataset	The full collection of data used	A table of 50,000 customers
Observation / Sample / Record / Instance	One row — four words for the same thing	One customer's record
Feature / Variable / Predictor / Input / Independent variable	A column used as input — five words for the same thing	`tenure_months`
Target / Label / Output / Dependent variable / Response	The column being predicted	`churn`
Training data	Data used to fit the model	70% of rows
Validation data	Data used to tune and compare models	15% of rows
Test data	Data used *once* for final, unbiased evaluation	15% of rows
Algorithm	The *procedure* for learning	"Logistic regression"
Model	The *result* of running an algorithm on data	The fitted equation with specific numbers in it
Parameter	A number *learned from data* during training	The weight on `tenure_months`
Hyperparameter	A setting *chosen by you before* training	Tree depth, learning rate, k in KNN
Training / Learning	The process of fitting parameters	Running `.fit()`
Prediction / Inference	Using the trained model on new data	Scoring today's customers
Generalization	Performing well on unseen data	Part 20
Classification	Predicting a category	Churn / no churn
Regression	Predicting a number	Expected revenue
Clustering	Grouping without labels	Customer segments
Representation	How data is encoded for the model	One-hot vectors, embeddings
Embedding	A learned, dense numeric vector representing an item	A 300-dimensional vector for a word or product
Loss function	Measures error on *one*	Squared error $(\hat{y} - y)^2$

Term	Meaning	Example
	example (or a batch)	
Cost function	Loss aggregated over the dataset (often used interchangeably with loss)	Mean squared error
Objective function	The quantity being optimized (cost, possibly plus penalties)	Cost + regularization
Optimization	The search for parameters minimizing the objective	Gradient descent
Epoch	One complete pass through the training data	10 epochs = 10 passes
Batch	A subset of data processed in one update	32 rows
Iteration	One parameter update	1,000 rows / 32 batch = ~32 iterations per epoch
Weight	A parameter multiplying a feature	0.34 × `tenure_months`
Bias (parameter)	The constant/intercept term	The `+ b` in `wx + b`
Bias (statistical)	Systematic error from oversimplification	Part 21 — *a different meaning of the same word*
Decision boundary	The surface separating predicted classes	The line where churn probability = 0.5
Probability	Model's estimated likelihood	`P(churn) = 0.83`
Score	Any numeric output used for ranking	A risk score
Threshold	The cutoff converting a probability to a decision	Flag if `P > 0.7`
Error	Difference between prediction and truth	Predicted 100, actual 120 → error 20
Residual	The error term in regression specifically	`y − ŷ`
Ground truth	The actual, verified answer	The customer really did churn

Two terminology traps beginners hit constantly:

1. **Algorithm vs Model.** The algorithm is the recipe; the model is the cake. "Random Forest" is an algorithm; the specific fitted forest sitting in your `.pkl` file is a model.
2. **Parameter vs Hyperparameter.** If the machine learns it from data, it's a parameter. If you set it before training starts, it's a hyperparameter. Tree *depth* is a hyperparameter; the *split values inside* the tree are parameters.

3. **"Bias" has two unrelated meanings** (the intercept term, and systematic underfitting error). Context disambiguates; the collision is unfortunate but universal.

---

7. Features

7.1 What a Feature Is

A feature is a measurable property of an observation used as model input. The quality of your features determines the ceiling on your model's performance far more than your choice of algorithm — a fact experienced practitioners internalize early and beginners discover painfully.

7.2 Feature Types

Type	Definition	Examples
Numerical	Quantities	`age`, `income`
Continuous	Numeric, any value in a range	`monthly_spend = 431.72`
Discrete	Numeric, countable	`num_transactions = 14`
Categorical	Membership in a category	`country`, `contract_type`
Nominal	Categorical, **no** order	`country`, `payment_method`
Ordinal	Categorical, **with** order	`satisfaction: low < medium < high`; `credit_rating: AAA > AA > A`
Binary	Two values	`has_paperless_billing`
Text	Free-form language	Support ticket body, product review
Image	Pixel arrays	X-ray, pipeline inspection photo
Audio	Waveforms	Call-centre recordings
Time-series	Values ordered in time	Daily sensor readings, monthly sales
Geographic	Location	Latitude/longitude, postcode, region
Derived / Engineered	Computed from other features	`debt_to_income = debt / income`

Why ordinal vs nominal matters practically: encoding a nominal variable (`country`) as integers 1,2,3 falsely tells the model that country 3 is "greater than" country 1 and that the distance from 1 to 3 is twice the distance from 1 to 2 — a fabricated relationship. Ordinal variables *can* legitimately be integer-encoded because the order is real. This single distinction prevents a very common beginner bug.

7.3 Worked Example — Customer Dataset

Column	Type	Feature or not?	Reasoning
`customer_id`	Identifier	**Not a feature**	A unique ID carries no generalizable signal; including it invites memorization and leakage
`age`	Numerical (discrete)	Feature	Plausibly relates to behaviour
`income`	Numerical (continuous)	Feature	Strong economic signal
`tenure`	Numerical (discrete)	Feature	Longer tenure typically means lower churn
`number_of_transactions`	Numerical (discrete)	Feature	Engagement proxy
`country`	Categorical (nominal)	Feature	Market differences are real
`occupation`	Categorical (nominal)	Feature	Income stability proxy
`account_type`	Categorical (nominal)	Feature	Product mix matters
`customer_status`	Categorical	**Depends — inspect carefully**	If it means "active/churned", this **is the target**, not a feature. Using it as a feature would be textbook leakage (Part 18)

That last row is the lesson: **you cannot classify a column as a feature or a target by its name — only by understanding what it means and when it becomes known.** A column recorded *after* the outcome you are predicting is never a valid feature.

7.4 Derived Features — Where Domain Knowledge Pays

```
# Raw columns often carry less signal than ratios and trends built
from them
df["debt_to_income"]      = df["total_debt"] / df["annual_income"]
df["spend_per_month"]     = df["total_spend"] / df["tenure_months"]
df["support_call_rate"]   = df["support_calls"] / df["tenure_months"]
df["spend_trend"]         = df["spend_last_3m"] / df["spend_prior_3m"]
```

Line	Why it helps
`debt_to_income`	Absolute debt is meaningless without income context; the *ratio* is what lenders actually

Line	Why it helps
	assess
`spend_per_month`	Normalizes for how long the customer has been around, making new and old customers comparable
`support_call_rate`	Ten calls in ten years is fine; ten calls in one month is a churn alarm
`spend_trend`	A *declining* trend is a far stronger churn signal than the absolute level

8. Target / Label

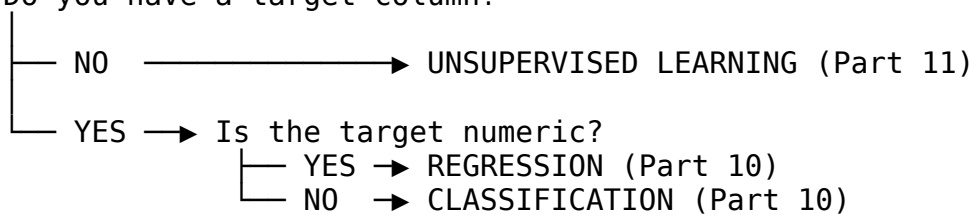
8.1 Definitions

The **target** (also **label**, **dependent variable**, **response variable**, **output**, conventionally written **y**) is the quantity the model predicts. It is "dependent" because we model it as depending on the features.

8.2 Examples

Problem	Target	Type	Resulting ML problem
House price prediction	`price`	Numeric, continuous	Regression
Churn prediction	`churn`	Binary category	Binary classification
Fraud detection	`is_fraud`	Binary category	Binary classification (imbalanced)
Loan risk	`default_status`	Binary category	Binary classification
Product category assignment	`category`	Multi-class category	Multiclass classification
Demand forecasting	`units_sold`	Numeric	Regression / time-series
Customer segmentation	**none**	—	Unsupervised (clustering)

8.3 Why the Target Determines Everything



This is the single most useful decision rule in this module, expanded into a full framework in Part 30.

8.4 Defining the Target Well — A Professional Skill

The target is rarely handed to you cleanly. Consider "churn" for a telecom:

- **Explicit churn:** the customer called and cancelled. Clean, but misses silent attrition.
- **Silent churn:** no usage for 60 days. Requires choosing 60 — why not 30 or 90?
- **Contract non-renewal:** only meaningful for contract customers.

Each definition yields a different label column, a different class balance, a different model, and a different business interpretation. **Getting the target definition right is a business conversation, not a technical one** — and it must happen before any modelling.

Prediction horizon matters equally. "Will this customer churn?" is unanswerable; "will this customer churn *within the next 30 days*?" is a well-posed learnable question. Every classification target on a future event needs an explicit time horizon.

---

9. Types of Machine Learning

9.1 The Five Paradigms

Paradigm	Data required	What it learns	Canonical examples
Supervised	Features + labels	A mapping f(X) → y	Fraud detection, price prediction
Unsupervised	Features only	Structure within X	Segmentation, anomaly detection
Semi-supervised	Many unlabelled + few labelled	A mapping, leveraging unlabelled data	Medical imaging with scarce expert labels
Self-supervised	Unlabelled data (labels generated from the data itself)	Reusable representations	LLMs, embeddings, foundation models
Reinforcement	An environment giving rewards	A policy: state → action	Game playing, robotics, dynamic pricing

9.2 Comparison Table

Dimension	Supervised	Unsupervised	Semi-supervised	Self-supervised	Reinforcement
Labels needed	Yes, all	None	Some	None (auto-generated)	No labels; rewards instead
Objective	Predict y	Find	Predict y	Learn	Maximize

Dimension	Supervised	Unsupervised	Semi-supervised	Self-supervised	Reinforcement
		structure	cheaply	representations	cumulative reward
Feedback	Immediate, per-example	None	Partial	From the data itself	Delayed, sparse
Evaluation	Straightforward (vs ground truth)	Difficult (no ground truth)	Standard supervised metrics	On downstream tasks	Cumulative reward
Typical algorithms	Linear/ logistic regression, trees, gradient boosting, neural nets	k-means, DBSCAN, PCA, isolation forest	Self-training, label propagation	Masked language modelling, contrastive learning	Q-learning, policy gradients
Business maturity	Very high — the workhorse	High	Moderate	High (via pretrained models)	Lower in most enterprises
Main limitation	Labels are expensive	Results are hard to validate	Assumptions can be wrong	Needs vast data and compute	Needs a safe environment to explore in

Realistic expectation setting: in most enterprises, **supervised learning accounts for the large majority of production ML value**, with unsupervised methods used mainly for segmentation and anomaly detection, self-supervised consumed indirectly via pretrained models and APIs, and reinforcement learning rare outside specific domains. A beginner should master supervised learning first, deeply.

10. Supervised Learning

10.1 The Setup

```
DATA:   X = features (the inputs)        y = target (the known
answers)
GOAL:   learn f such that  f(X) ≈ y
USE:    given new X_new, output ŷ = f(X_new)
```

"Supervised" refers to the labels acting as a supervisor: for every training example the algorithm is *told* the right answer, measures how wrong it was, and adjusts.

10.2 A Concrete Walkthrough

tenure_months	monthly_spend	support_calls	churn (y)
2	20	5	1
36	85	0	0
5	25	4	1
48	120	1	0

The algorithm searches for a function mapping the three feature columns to the churn column. It might discover something like: *low tenure + low spend + many support calls → high churn probability*. That discovered relationship, expressed as numbers, is the model.

10.3 Regression — Predicting a Number

Definition: supervised learning where the target is continuous and numeric.

Application	Target	Typical features
House price prediction	`price` (₦/$)	Size, location, bedrooms, age
Revenue prediction	`next_quarter_revenue`	Pipeline, seasonality, headcount
Demand forecasting	`units_sold`	History, price, promotion, weather
Salary prediction	`salary`	Experience, role, location, education
Energy consumption	`kwh_consumed`	Temperature, day type, occupancy
Oil production forecasting	`barrels_per_day`	Reservoir pressure, well age, choke size

How you evaluate it: by how *far off* you are — Mean Absolute Error, Root Mean Squared Error, R^2. Covered fully in the evaluation module.

10.4 Classification — Predicting a Category

Definition: supervised learning where the target is categorical.

Application	Target	Note
Fraud detection	fraud / not fraud	Severely imbalanced
Churn prediction	churn / no churn	Moderately imbalanced
Spam filtering	spam / ham	False positives are costly
Disease screening	disease / no disease	False negatives are costly
Loan default	default / no default	Regulated, explainability required

How you evaluate it: accuracy, precision, recall, F1, ROC-AUC — and choosing correctly among them is a business decision, not a technical one (Part 28).

10.5 Binary, Multiclass, Multilabel

Type	Definition	Example
Binary	Exactly two mutually exclusive classes	Fraud / not fraud
Multiclass	More than two, **mutually exclusive** — each row gets exactly one	Product category: Electronics *or* Apparel *or* Home
Multilabel	Multiple labels can apply **simultaneously** to one row	A support ticket tagged *billing* **and** *urgent* **and** *escalated*

The multiclass/multilabel distinction is frequently confused and materially changes model setup and metrics. Test: *can one observation legitimately belong to two classes at once?* If yes, it's multilabel.

10.6 Advantages and Limitations

Advantages: clear objective, straightforward evaluation against ground truth, mature tooling, the paradigm behind most production ML value.

Limitations: requires labelled data (often expensive or slow to obtain — a fraud label may only be confirmed 60 days after the transaction); performance is capped by label quality; assumes future data resembles training data.

---

11. Unsupervised Learning

11.1 The Setup

```
DATA:   X = features only (no y)
GOAL:   discover structure within X
OUTPUT: groupings, compressed representations, anomaly scores, or
rules
```

11.2 Clustering

Grouping observations so that members of a group are more similar to each other than to members of other groups.

Business example — retail customer segmentation. Input: recency, frequency, monetary value, category mix. Output: five clusters. Interpretation is a human job: the analyst examines cluster centroids and names them — "high-value loyalists", "discount-driven occasionals", "lapsing mid-tier". Decision: different marketing treatment per segment.

The key difficulty: there is no ground truth. Nothing tells you that five clusters is right rather than four or seven, and nothing tells you the clusters are *meaningful* rather than arbitrary partitions of a continuous blob. Validation is partly statistical, largely judgement.

11.3 Dimensionality Reduction

Compressing many features into fewer while retaining most of the information. Used for visualization (projecting 50 dimensions to 2 for plotting), for speeding up downstream models, and for removing redundancy among correlated features. PCA is the canonical method.

11.4 Anomaly Detection

Identifying observations that differ markedly from the norm.

Business example — cybersecurity. Input: network traffic features. The model learns what normal traffic looks like and flags deviations. Crucially, this works **without labelled attacks** — valuable precisely because novel attacks have no historical examples by definition. Also widely used in manufacturing (equipment fault detection) and finance (unusual transaction patterns).

Supervised vs unsupervised fraud detection: supervised catches *known* fraud patterns well but is blind to genuinely new ones; unsupervised anomaly detection catches novelty but generates more false positives. Mature institutions run **both**.

11.5 Association Rule Learning

Finding co-occurrence rules: *customers who buy X also buy Y*. Classic market-basket analysis. Output is human-readable rules with support and confidence measures — used for store layout, bundling, and cross-sell recommendations.

11.6 Supervised vs Unsupervised on the Same Dataset

Same customer table, two different projects:

	Supervised	Unsupervised
Question	"Which customers will churn next month?"	"What natural customer types exist?"
Requires	A `churn` column	No target
Output	A probability per customer	A segment label per customer
Validation	Compare predictions to what	Judgement, stability checks,

	Supervised	Unsupervised
	actually happened	business usefulness
Success	Measurable in retained revenue	Measurable only indirectly

12. Semi-Supervised Learning

12.1 The Setup

```
Large amount of UNLABELLED data  +  Small amount of LABELLED data
                              ↓
                  Semi-supervised learning
```

12.2 Why Labelling Is Expensive

Labels frequently require scarce expert time (a radiologist annotating scans), long waiting periods (a loan's default status is unknown for months or years), costly investigation (confirming a transaction was fraudulent requires a manual case review), or simply enormous manual effort. Meanwhile *unlabelled* data — the raw scans, transactions, documents — accumulates cheaply and abundantly. Semi-supervised learning exists to exploit that asymmetry.

12.3 How It Helps

The core intuition: unlabelled data reveals the *shape* of the data distribution — where the dense regions are, where the natural boundaries lie — even though it says nothing about labels. If the decision boundary is likely to fall in a low-density region between clusters, unlabelled data helps locate it, so a handful of labels can go much further than they otherwise would.

Common approaches include **self-training** (train on the labelled data, predict on the unlabelled data, add the most confident predictions to the training set as pseudo-labels, repeat) and **label propagation** (spread labels through a similarity graph connecting similar examples).

The central risk: self-training amplifies its own mistakes. A confidently wrong pseudo-label becomes training data, reinforcing the error. Confidence thresholds and careful validation are essential.

12.4 Practical Examples

Domain	Situation
Medical imaging	200 expert-annotated scans, 100,000 unannotated ones
Document classification	A few hundred manually categorized contracts, millions in the archive
Fraud detection	Confirmed fraud cases are scarce; transactions are abundant
Speech recognition	Transcribing audio is slow; raw audio is

Domain	Situation
	plentiful

13. Self-Supervised Learning

13.1 The Core Idea

Self-supervised learning creates **labels automatically from the structure of the data itself**, so no human annotation is needed at all — yet the training is technically supervised.

The canonical example: take a sentence, hide a word, and train the model to predict the hidden word.

```
Original:  "The customer cancelled their subscription after the price
increase."
Input:     "The customer cancelled their [MASK] after the price
increase."
Target:    "subscription"
```

Nobody labelled anything — the label *was already in the data*; we simply removed it and asked the model to restore it. Repeat across billions of sentences and the model is forced to learn grammar, semantics, factual associations and reasoning patterns, because all of that is required to fill blanks well.

13.2 Pretext Tasks and Representation Learning

The fill-in-the-blank task is a **pretext task** — we don't actually care about predicting masked words. We care about the internal **representations** the model develops while learning to do it. Those representations transfer to tasks we *do* care about: classification, search, summarization.

This two-stage pattern is the defining architecture of modern AI:

```
STAGE 1 — PRE-TRAINING (self-supervised, enormous data, enormous
compute, done once)
    Raw text/images  →  pretext task  →  a model with rich general
representations
                                                 |
STAGE 2 — FINE-TUNING / ADAPTATION (supervised, small data, cheap,
done many times)
                                                 ↓
                          Your specific task, with a few hundred or
few thousand labels
```

13.3 Connecting to BERT, GPT, Embeddings and Foundation Models

System	Self-supervised objective
BERT-style models	Masked language modelling — predict hidden words using context from *both* directions. Strong for understanding tasks

System	Self-supervised objective
	(classification, extraction)
GPT-style models	Next-token prediction — predict the following word given everything before it. Naturally suited to *generation*
Embeddings	The dense vectors produced by these models; semantically similar items land near each other, which is what powers semantic search and RAG
Contrastive learning	Train the model to pull representations of related items together and push unrelated ones apart — heavily used in vision and in embedding models
Foundation models	The general term for large pre-trained models adaptable to many downstream tasks

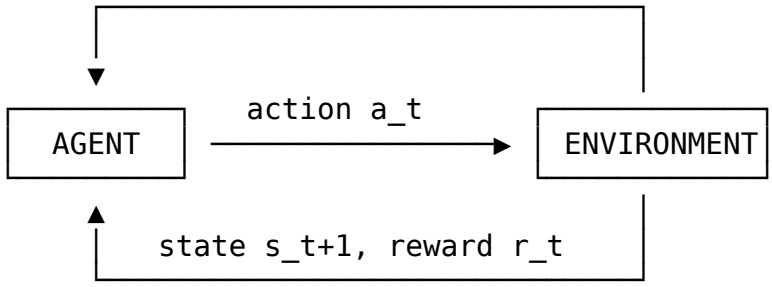
Why this matters to a beginner right now: you will almost never pre-train a foundation model — that requires resources only a handful of organizations have. You will constantly *use* one: calling an embedding API, fine-tuning a small model, or building on an LLM. Understanding that these models were built by self-supervised learning explains both their broad capability and their characteristic weaknesses (they learned statistical regularities of their training corpus, including its errors and biases — Part 40).

---

14. Reinforcement Learning

14.1 The Setup

Reinforcement learning studies an **agent** learning to act in an **environment** through trial, error and reward.



14.2 Core Vocabulary

Term	Meaning	Chess example
Agent	The learner/decision-maker	The player
Environment	Everything the agent interacts with	The board and opponent

Term	Meaning	Chess example
State	The current situation	The current board position
Action	A choice available to the agent	A legal move
Reward	Immediate numeric feedback	+1 for winning, −1 for losing, 0 otherwise
Policy	The agent's strategy: state → action	"In this position, play this move"
Value	Expected *total future* reward from a state	"This position is winning"
Episode	One complete run from start to terminal state	One full game

14.3 What Makes RL Fundamentally Different

In supervised learning, every example comes with the correct answer. In RL, the agent is never told the right action — only how good the outcome was, often **long after** the decisive action was taken. A chess move made at turn 12 may determine a loss at turn 60. Assigning credit across that gap (the *credit assignment problem*) is RL's central difficulty.

RL also faces the **exploration-exploitation tradeoff**: should the agent take the action it currently believes is best (exploit), or try something unfamiliar that might be better (explore)? Too much exploitation locks in a mediocre strategy; too much exploration never converges.

14.4 Examples

Application	Agent	Reward
Game playing	The player	Win/loss
Robot navigation	The robot	Reaching the goal without collision
Recommendation	The recommender	Clicks, watch time, long-term retention
Resource allocation	A scheduler	Throughput, cost reduction
Dynamic pricing	Pricing engine	Revenue or margin

Why RL is rarer in enterprises than its fame suggests: it needs an environment safe to explore in. A bank cannot let an agent "explore" by randomly approving risky loans to learn from the outcomes. Where a high-fidelity simulator exists (games, logistics, some industrial control), RL is powerful; where it doesn't, supervised learning usually wins on practicality. Advanced RL mathematics belongs to a later module.

---

15. Batch vs Online Learning

15.1 Definitions

Batch (offline) learning: the model is trained on the complete dataset at once, then deployed as a fixed artefact. Updating means retraining from scratch on a schedule.

Online (incremental) learning: the model updates continuously as each new example or mini-batch arrives.

15.2 Comparison

Dimension	Batch	Online
Data processing	All at once	Sequentially, as it arrives
Training frequency	Periodic (nightly, weekly, monthly)	Continuous
Memory	Must hold (or stream) the full dataset	Only the current example and model state
Adaptability	Lags by one retraining cycle	Adapts immediately
Stability	High, predictable, easy to validate before release	Can degrade quickly if fed bad data
Reproducibility	Strong — a specific model version is a fixed artefact	Harder — the model is a moving target
Testing before release	Straightforward	Difficult
Typical use	Most enterprise ML	Streaming, rapid-drift, huge-scale settings

15.3 Examples

- **Monthly churn model** → batch. Churn evolves slowly; monthly retraining is ample; stability and auditability matter more than immediacy.
- **Real-time fraud detection** → *hybrid in practice*: the model is usually trained in batch (for stability and validation) but **serves** in real time with fresh features computed at request time. True online weight updates are less common than beginners assume, precisely because an attacker who can influence training data could poison a continuously-updating model.
- **Recommendation systems** → often hybrid: batch-trained base model with online updating of recency/context signals.
- **Streaming sensor data (IoT/energy)** → genuinely online where patterns shift fast and data volume prohibits storage.

A distinction beginners blur: *batch vs online* refers to **training**, not serving. A batch-trained model can absolutely serve real-time predictions at millisecond latency. Separate the two questions: how often does the model *learn*, and how fast must it *answer*?

---

16. Instance-Based vs Model-Based Learning

16.1 Instance-Based Learning

The system **memorizes training examples** and makes predictions by comparing new inputs to stored ones. There is no compact learned function — the data *is* the model.

Example — K-Nearest Neighbours (KNN), conceptually: to predict whether a new customer will churn, find the *k* most similar customers in the training data (by whatever distance measure), and take a vote among their outcomes. If 7 of the 10 most similar past customers churned, predict churn.

16.2 Model-Based Learning

The system **builds a compact model** — a function with parameters — that summarizes the patterns, then discards the training data. Prediction applies the function.

Example — Linear regression, conceptually: learn a weight per feature so that predicted price = $w_1 \cdot$size + $w_2 \cdot$bedrooms + … + b. Once trained, the original sales data is no longer needed; the handful of weights *is* the model.

Example — Decision trees, conceptually: learn a sequence of yes/no questions on features, arranged as a tree. Prediction walks the tree from root to leaf.

16.3 Comparison

Dimension	Instance-based	Model-based
How it predicts	Compares to stored examples	Applies a learned function
Training cost	Very low (often just storing data)	Higher (optimization)
Prediction cost	High (must search the training set)	Very low
Storage	Entire training set	Just parameters
Generalization	Local — depends on neighbourhood structure	Global — the learned function applies everywhere
Interpretability	"These similar cases churned" — intuitive per-case	Depends on model; linear models very interpretable
Scaling to big data	Poor	Good
Examples	KNN, kernel methods	Linear/logistic regression, trees, neural networks

Practical implication: instance-based methods flip the usual cost profile — cheap to train, expensive to serve. That is often the wrong tradeoff for production systems needing low-latency predictions at scale, which is a major reason KNN is more common as a teaching example and baseline than as a deployed model. Algorithm mathematics comes in later modules.

---

17. Parametric vs Non-Parametric Models

17.1 The Distinction

Parametric models assume a specific functional form and have a **fixed number of parameters** regardless of dataset size. Linear regression on 10 features has 11 parameters whether you train on 1,000 rows or 100 million.

Non-parametric models make **no fixed assumption about functional form**, and their effective complexity **grows with the data**. A decision tree can grow deeper with more data; KNN's "model" is the entire dataset.

Terminology warning: "non-parametric" does *not* mean "no parameters" — a deep decision tree has many. It means the *number* of parameters is not fixed in advance by the model's form.

17.2 Comparison

Dimension	Parametric	Non-parametric
Assumptions	Strong (e.g. the relationship is linear)	Weak/few
Number of parameters	Fixed	Grows with data
Data requirements	Works reasonably with less data	Generally needs more data
Flexibility	Limited to the assumed form	Can fit complex, arbitrary shapes
Risk profile	Higher bias, lower variance → underfitting risk (Part 21)	Lower bias, higher variance → overfitting risk
Speed	Fast to train and predict	Often slower
Interpretability	Usually high	Usually lower
Examples	Linear regression, logistic regression, naive Bayes	Decision trees, random forests, KNN, gradient boosting

17.3 Intuition

Parametric is like assuming a relationship is a straight line and only asking "which straight line?" — a constrained but efficient search. Non-parametric is like allowing any curve at all — far more expressive, but requiring much more data to pin down, and far easier to fit to noise.

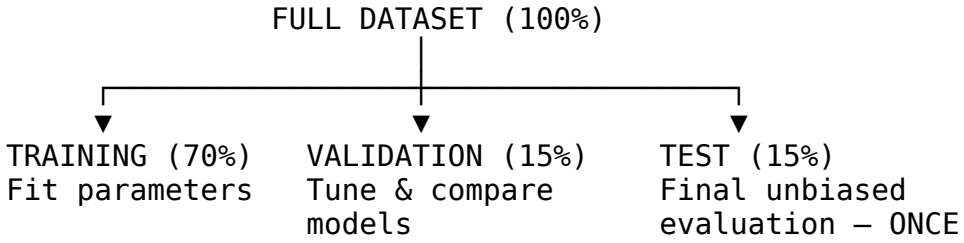
Practical guidance for beginners: start with a simple parametric model as your baseline (Part 39). If it underfits badly, the extra flexibility of a non-parametric model is justified. Reaching for maximum flexibility first is a common mistake (Part 28) that produces models which look excellent in training and fail in production.

18. Training, Validation and Testing

18.1 Why Data Must Be Divided

If you evaluate a model on the same data it learned from, you measure **memorization**, not **learning**. A model that stores every training row and looks up answers would score 100% — and be useless on a new customer. The only honest estimate of future performance comes from data the model has never seen.

18.2 The Three Sets



Set	Purpose	How often used
Training	The model learns its parameters from this	Continuously during training
Validation	Compare candidate models, tune hyperparameters, decide when to stop	Many times during development
Test	Estimate real-world performance of the final chosen model	**Once**, at the very end

Why the validation set is separate from the test set: every time you look at a result and change something, you leak a little information about that data into your decisions. After fifty rounds of "try a setting, check validation score, adjust," your validation score has become optimistically biased — it partly reflects choices tuned *to that specific set*. The test set, untouched throughout, is the only clean estimate remaining.

18.3 Common Splits

Split	When appropriate
70 / 15 / 15	A reasonable general default
80 / 20 (train/test) with **cross-validation** on the training portion	Very common; cross-validation replaces a fixed validation set
60 / 20 / 20	Larger validation and test sets when data is plentiful
98 / 1 / 1	Very large datasets — 1% of 10 million rows is 100,000 rows, ample for evaluation

Cross-validation (introduced here, detailed in the evaluation module): split the training data into k folds, train on $k–1$ and validate on the held-out fold, rotate, and average. It uses data

more efficiently and gives a more stable estimate than a single validation split — valuable especially on smaller datasets.

18.4 Splitting Strategies

Strategy	How it works	When to use
Random	Rows assigned randomly	Independent observations, balanced classes
Stratified	Random, but preserving class proportions in each split	**Imbalanced classification** — essential. With 1% fraud, a naive random split can leave a test set with almost no fraud cases
Time-based	Train on earlier periods, test on later ones	**Any temporal data**: forecasting, churn, fraud. Non-negotiable
Group-based	All rows for a given entity stay in one split	When multiple rows share an entity — several transactions per customer, several scans per patient

Time-based splitting deserves emphasis. If you randomly split time-series data, the model trains on future data and is evaluated on past data — an impossible situation in production. The resulting scores will be excellent and entirely fictitious. This is one of the most damaging beginner errors (Part 28).

Group-based splitting matters equally: if customer A has 50 transactions split randomly across train and test, the model effectively sees that customer during training and is then tested on the same customer — inflating scores, because in production the customer will often be genuinely new.

18.5 Data Leakage

Definition: data leakage occurs when information unavailable at real prediction time influences training, producing performance estimates that cannot be reproduced in production.

Leakage type	Example	Symptom
Target leakage	Including `account_closed_date` when predicting churn	Suspiciously perfect accuracy
Train-test contamination	Scaling/imputing using statistics computed on the *entire* dataset before splitting	Modest but consistent over-optimism
Temporal leakage	Random splitting of time-ordered data	Excellent test scores, poor production performance
Group leakage	Same customer in train and test	Inflated scores

Leakage type	Example	Symptom
Duplicate rows	The same record in both splits	Inflated scores

The professional rule: *split first, then do everything else.* Fit imputers, scalers and encoders on the training set only, then apply those fitted transformations to validation and test. Scikit-learn `Pipeline` objects exist largely to enforce this automatically.

The diagnostic heuristic: if your model looks far better than you expected, assume leakage until proven otherwise. Genuine breakthroughs are rare; leakage is common.

---

19. What Does "Learning" Mean Mathematically?

19.1 The Notation

Symbol	Meaning
X	The features (inputs)
y	The true target
θ (theta)	The model's parameters — the numbers it learns
f_θ(X)	The model: a function of X, controlled by θ
ŷ ("y-hat")	The model's prediction, ŷ = f_θ(X)
L(θ)	The loss: how wrong the model is, as a function of its parameters

19.2 The Central Equation, In Words

```
θ* = argmin_θ  L(θ)
```

Read aloud: *"The best parameters, θ-star, are the particular values of θ that make the loss L as small as possible."*

`argmin` simply means "the argument that minimizes" — not the minimum value itself, but the *setting that achieves it*. If loss is lowest when the weight equals 2.5, then argmin is 2.5.

That single line is what training is. Everything else — gradient descent, backpropagation, learning rates — is machinery for *finding* that minimum efficiently.

19.3 A Fully Worked Numerical Example

Predict monthly spend from tenure using the simplest possible model: **ŷ = w × tenure** (one parameter, no intercept, for clarity).

Training data:

tenure	actual spend (y)
1	12

tenure	actual spend (y)
2	19
3	31

Try w = 5:

tenure	ŷ = 5 × tenure	y	error (ŷ − y)	squared error
1	5	12	−7	49
2	10	19	−9	81
3	15	31	−16	256

Mean squared error = (49 + 81 + 256) / 3 = **128.67**

Try w = 10:

tenure	ŷ	y	error	squared error
1	10	12	−2	4
2	20	19	+1	1
3	30	31	−1	1

Mean squared error = (4 + 1 + 1) / 3 = **2.00**

Try w = 12:

tenure	ŷ	y	error	squared error
1	12	12	0	0
2	24	19	+5	25
3	36	31	+5	25

Mean squared error = (0 + 25 + 25) / 3 = **16.67**

w	Loss
5	128.67
10	**2.00** ← best so far
12	16.67

The loss falls then rises — so the optimum lies near w = 10. **That search is learning.** A real algorithm doesn't guess randomly; it uses the *slope* of the loss to decide which direction to move and by how much (gradient descent), but the logic is exactly what you just did by hand.

19.4 Why Errors Are Squared

Squaring makes all errors positive (so +5 and −5 don't cancel out to zero and falsely suggest a perfect model), and penalizes large errors disproportionately — an error of 10 contributes 100, while two errors of 5 contribute 50 total. Whether that disproportionate penalty is desirable is a

modelling choice; alternatives like absolute error treat all errors proportionally and are more robust to outliers.

19.5 Putting It Together

```
1. Start with arbitrary parameters θ
2. Predict:        ŷ = f_θ(X)
3. Measure error:  L(θ) = how far ŷ is from y
4. Adjust θ in the direction that reduces L
5. Repeat until L stops improving meaningfully
6. The resulting θ* is your trained model
```

No calculus was required to understand this. The calculus (derivatives telling you the direction and steepness of the loss surface) arrives in a later module — but the *concept* above is the entirety of what "training a model" means.

---

20. Generalization

20.1 The Definition

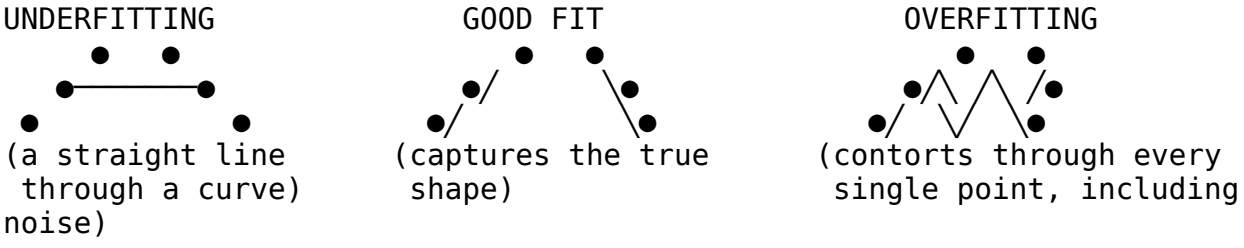
Generalization is a model's ability to perform well on data it has never seen. It is the entire objective of machine learning. A model that performs perfectly on training data and poorly on new data has failed completely, regardless of how impressive the training numbers look.

20.2 Memorization vs Pattern Learning

Memorization stores specific examples. **Pattern learning** extracts the underlying regularity that will still hold for new examples.

A student who memorizes that last year's exam question 3 answer was "B" learns nothing transferable. A student who understands the concept can answer a differently-worded question. Models face the same distinction, and — critically — a sufficiently complex model *can always memorize* a finite training set perfectly. Its training score then tells you nothing about its usefulness.

20.3 Overfitting and Underfitting




```
Train error:  HIGH          LOW                VERY LOW
Test error:   HIGH          LOW                HIGH
```

	Underfitting	Overfitting
What happened	The model is too simple to	The model learned noise and

	Underfitting	Overfitting
	capture the real pattern	quirks specific to the training data
Training performance	Poor	Excellent
Test performance	Poor	Poor
Gap between them	Small (both bad)	Large
Typical causes	Too-simple model; too few features; over-restricted	Too-complex model; too many features; too little data; trained too long
Direction of fix	Add complexity/features	Reduce complexity, get more data, regularize (later module)

20.4 The Diagnostic Table

This table is worth memorizing; it is how practitioners diagnose a model in seconds:

Training score	Test score	Diagnosis	Action
Low	Low	**Underfitting**	More complex model, better features
High	High	**Good fit**	Ship it
High	Low	**Overfitting**	Simplify, regularize, more data
Low	High	**Suspicious**	Usually a bug, a broken split, or an unrepresentative test set

Regularization — the main technical toolkit for combating overfitting — is introduced properly in a later module.

---

21. Bias and Variance

21.1 Definitions

Bias is error from **wrong assumptions** — the model is too simple to represent reality. High bias → underfitting.

Variance is error from **sensitivity to the specific training data** — retrain on a slightly different sample and you get a noticeably different model. High variance → overfitting.

21.2 The Archery Intuition

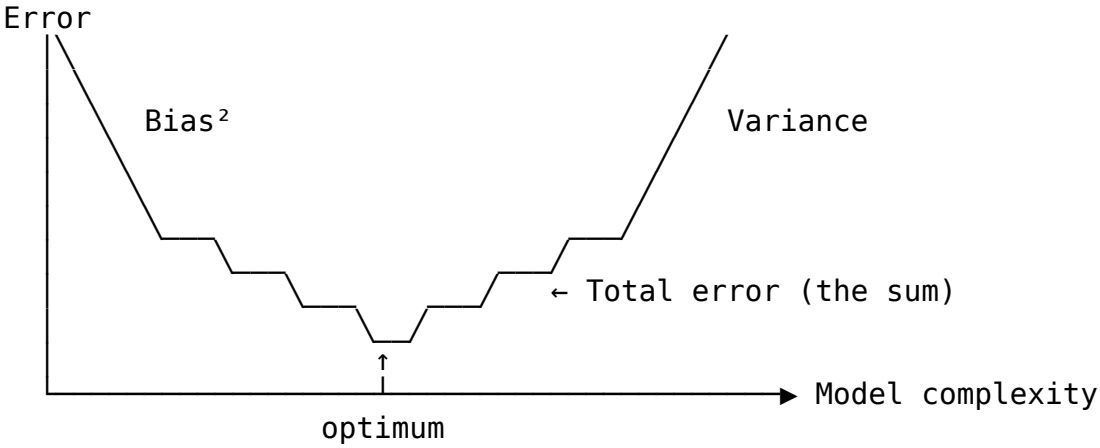
Imagine shooting at a target repeatedly, where each "shot" is a model trained on a different random sample of data:

```
LOW BIAS, LOW VARIANCE          HIGH BIAS, LOW VARIANCE
   (the goal)                    (consistently wrong)
        ⊕⊕
       ⊕⊕⊕   ← on target                ⊕⊕⊕
                                        ⊕⊕⊕   ← tight group, wrong place

LOW BIAS, HIGH VARIANCE         HIGH BIAS, HIGH VARIANCE
 (right on average,              (the worst case)
  wildly inconsistent)
   ⊕      ⊕                          ⊕
      ⊕⊕                                ⊕
   ⊕      ⊕                     ⊕              ⊕
```

Bias = how far the *average* shot is from the bullseye. Variance = how *spread out* the shots are.

21.3 The Tradeoff



As complexity increases, bias falls (the model can represent more) but variance rises (it can chase noise). Total error is minimized somewhere in the middle — **finding that point is much of practical machine learning.**

21.4 Practical Signals and Remedies

Symptom	Likely problem	Remedies
Both train and test scores poor	**High bias**	More complex model, better/more features, train longer, reduce regularization
Train score excellent, test score poor	**High variance**	Simpler model, more training data, regularization, feature selection, ensembling
Scores swing a lot across cross-validation folds	**High variance**	More data; more stable model class

A note on the word: “bias” here is a *statistical* property, entirely distinct from *social/ethical bias* in Part 40 (unfair treatment of groups). Both are real concerns; they are unrelated concepts that share a word.

22. ML vs Data Analytics vs Data Science vs Adjacent Disciplines

22.1 The Disciplines Compared

Discipline	Core question	Primary output	Core tools
Data Engineering	How do we get reliable data where it's needed?	Pipelines, warehouses, tables	SQL, Python, Spark, Airflow, dbt, cloud
Data Analytics	What happened, and why?	Dashboards, reports, analyses	SQL, Excel, Power BI/Tableau, Python
Data Science	What will happen, and what should we do?	Models, experiments, insights	Python, statistics, scikit-learn, notebooks
Machine Learning	How do we learn a function from data?	Trained models	Python, scikit-learn, XGBoost, PyTorch
ML Engineering	How do we run models reliably in production?	Serving systems, pipelines, monitoring	Python, Docker, Kubernetes, MLflow, cloud
AI Engineering	How do we build applications on foundation models?	LLM/RAG applications, agents	APIs, vector databases, orchestration frameworks
Software Engineering	How do we build correct, maintainable systems?	Applications and services	Language of choice, testing, CI/CD, architecture

Analytics is retrospective and diagnostic; data science is predictive and prescriptive; ML engineering is operational. They overlap heavily, and titles vary wildly between companies — the *work* is more informative than the label.

22.2 A Churn Project Across Roles

```
BUSINESS TEAM       "Churn is up 3 points. Can we predict who's at
risk?"
      ↓
DATA ENGINEER       Builds pipelines: billing, CRM, support, usage
                    → a clean, refreshed customer table in the
warehouse
      ↓
DATA ANALYST        Analyses historical churn: which segments, when,
correlates
                    → establishes the baseline and quantifies the
opportunity
      ↓
DATA SCIENTIST      Frames the ML problem, engineers features, trains
and
                    validates models, evaluates against the business
metric
```

```
                    → a validated model plus an interpretation of its
drivers
      ↓
ML ENGINEER         Productionizes: batch scoring pipeline or API,
monitoring,
                    retraining schedule, alerting
      ↓
AI/BACKEND ENGINEER Integrates scores into the CRM; optionally uses an
LLM to
                    draft personalized retention outreach (Part 24)
      ↓
BUSINESS TEAM       Runs the retention campaign; measures retained
revenue;
                    feeds outcomes back as new labels
```

The loop closes at the bottom: campaign outcomes become fresh training labels. Projects that never close this loop cannot improve.

22.3 What This Means for a Learner

You do not need all seven skill sets. You *do* need to understand the handoffs, because the most common cause of a failed ML project is not a bad model — it is a broken handoff: data that doesn't arrive reliably, a model nobody deploys, or predictions nobody acts on.

---

23. Machine Learning vs Deep Learning

23.1 The Fundamental Difference

Classical ML relies on **human-engineered features**. You decide that `debt_to_income` matters and compute it; the algorithm learns weights over the features you supplied.

Deep learning learns **its own features** from raw data. Given raw pixels, early layers learn edges, middle layers learn shapes, later layers learn objects — none of which anyone specified.

```
CLASSICAL ML
  Raw data → [HUMAN engineers features] → algorithm → model

DEEP LEARNING
  Raw data → [MODEL learns its own representations across layers] →
model
```

23.2 Comparison

Dimension	Classical ML	Deep Learning
Dataset size	Works well from thousands of rows	Typically needs very large data (or a pretrained model)
Feature engineering	Essential, human-driven, often decisive	Largely automatic
Compute	CPU is usually fine	GPUs/TPUs typically

Dimension	Classical ML	Deep Learning
		required for training
Training time	Minutes to hours	Hours to weeks
Interpretability	Often good (especially linear models, trees)	Poor by default; needs dedicated explainability methods
Best on	**Tabular/structured data**	**Unstructured data**: images, audio, text, video
Cost	Low	High
Maintenance	Simpler	More complex

23.3 Where Specific Methods Fit

Method	Family	Typical use
Linear / logistic regression	Classical, parametric	Baselines; regulated domains needing explainability
Decision trees	Classical, non-parametric	Interpretable rules; building block for ensembles
Random forests	Classical ensemble	Strong, robust general-purpose tabular models
XGBoost / LightGBM / CatBoost	Classical (gradient boosting)	**The standard winner on tabular business data**
Neural networks (MLPs)	Deep learning	General function approximation
CNNs	Deep learning	Images and spatial data
RNNs / LSTMs	Deep learning	Sequences (largely superseded by Transformers)
Transformers	Deep learning	Text, and increasingly everything else; the basis of LLMs

23.4 The Most Important Practical Point

For tabular business data — the overwhelming majority of enterprise ML — gradient-boosted trees usually match or beat deep learning, while training in minutes on a laptop, requiring less data, and offering better interpretability. Deep learning's dominance is real but concentrated in *unstructured* data.

A beginner who learns that "deep learning is the advanced/better option" will reach for a neural network on a 30,000-row customer table and be beaten by an XGBoost model that took two minutes to train. Choose the tool for the data type, not for prestige.

---

24. Machine Learning vs Generative AI

24.1 The Core Distinction

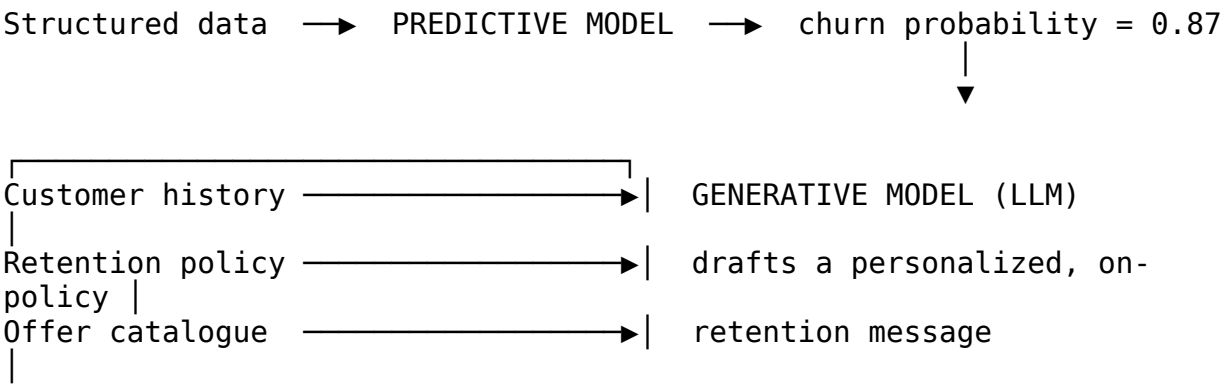
	Predictive ML	Generative AI
Output	A label, number or ranking	New content: text, image, code, audio
Question	"What will happen?" / "What is this?"	"Produce something appropriate"
Training	Usually supervised on labelled data	Usually self-supervised on vast corpora (Part 13)
Evaluation	Accuracy, RMSE, AUC — objective ground truth	Often subjective; needs human or model-based judging
Determinism	Same input → same output	Same input → *varying* outputs by default
Failure mode	Wrong prediction	Fluent, confident, **fabricated** content

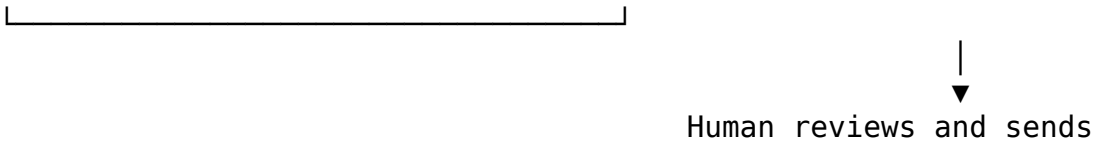
24.2 Paired Examples

Predictive ML	Generative AI
"Will this customer churn?" → 0.87	"Write a personalized retention message for this customer."
"Is this transaction fraudulent?" → 0.93	"Explain why this transaction pattern is suspicious, for the case file."
"Which support tickets are urgent?" → priority score	"Draft a response to this ticket."
"What is this document's category?" → *Contract*	"Summarize this contract's obligations and risks."

24.3 How They Work Together

The highest-value modern architectures use **both**, each for what it does well:





The predictive model decides **who** and **how urgently** — a task requiring calibrated numeric judgement over structured data, which LLMs do poorly. The generative model handles **what to say** — language production, which predictive models cannot do at all. Asking an LLM to estimate churn probability from a table is using the wrong tool; so is trying to make a gradient-boosted tree write an email.

The critical caveat: generative models produce fluent text regardless of whether it is true. Any generative output touching a customer, a regulator or a clinical decision requires verification, grounding in retrieved facts (RAG), or human review. This is a defining engineering constraint, covered in later modules.

25. Real-World Machine Learning Project

25.1 The Business Problem

Company: a subscription telecom operator with 50,000 active customers. **Situation:** monthly churn has risen to 4%, representing roughly ₦180 million in annualized lost revenue. Acquiring a new customer costs approximately 5× the cost of retaining an existing one. **Business objective:** identify customers likely to cancel within the next 30 days so the retention team — which has capacity to contact about 2,000 customers per month — can intervene. **Success metric (business):** retained revenue attributable to the campaign, net of offer costs. **Success metric (model):** because the team can only contact 2,000 of 50,000 (4%), what matters is **how many true churners appear in the top 4% by predicted risk** — a *ranking* problem. Raw accuracy is the wrong metric (Part 28).

25.2 The Dataset

Column	Meaning	Type
`customer_id`	Unique identifier	Identifier — **not a feature**
`age`	Customer age in years	Numeric (discrete)
`gender`	Gender	Categorical (nominal) — *sensitive attribute, see Part 40*
`country`	Country of residence	Categorical (nominal)
`tenure_months`	Months as a customer	Numeric (discrete)
`monthly_income`	Estimated monthly income	Numeric (continuous)
`monthly_spend`	Average monthly bill	Numeric (continuous)
`num_transactions`	Transactions in last month	Numeric (discrete)
`support_calls`	Support calls in last 3 months	Numeric (discrete)
`contract_type`	Month-to-month / One year / Two year	Categorical (ordinal-ish)
`payment_method`	Card / Bank transfer / Cash	Categorical (nominal)
`customer_satisfaction`	Survey score 1–5	Ordinal
`churn`	Did the customer cancel? (0/1)	**TARGET**

25.3 Conceptual Workflow

1. Business problem — reduce churn by enabling targeted retention (Section 25.1).

2. Data — one row per customer; 50,000 observations; a snapshot of features as of a cutoff date, with `churn` recorded over the following 30 days. *That temporal construction is essential:* features must come from **before** the prediction window, or you have leakage (Part 18).

3. Features — all columns except `customer_id` (an identifier with no generalizable signal) and `churn` (the target). `gender` requires a deliberate decision: including it may improve accuracy while creating fairness and regulatory exposure (Part 40).

4. Target — `churn`, binary, defined precisely as *"cancelled service within 30 days of the snapshot date."*

5. ML problem type — supervised (we have labels) + categorical target → **binary classification**. Likely imbalanced (~4% positive), so stratified splitting is required.

6. Train/test concept — because churn is temporal, the professionally correct split is **time-based**: train on customers as of January–September, test on October–December. A random split would let the model learn from future periods.

7. Model concept — start with a simple, interpretable baseline (Part 39), then a logistic regression, then gradient-boosted trees if justified.

8. Prediction concept — output a **probability** per customer (0.00–1.00), not a hard label. Probabilities enable ranking, which is what the campaign actually needs.

9. Evaluation concept — rank all customers by probability, take the top 2,000, and ask: what fraction of actual churners did we capture? (*recall at the operating point*), and what fraction of those contacted actually would have churned? (*precision at the operating point*). Compare against the baseline of contacting 2,000 customers at random.

10. Business decision — the retention team contacts the top-ranked 2,000 with a targeted offer. Outcomes are recorded and become next quarter's training labels.

25.4 Simple Python Implementation

```
import pandas as pd

# Load the dataset
df = pd.read_csv("customer_churn.csv")

# 1. Inspect structure
print(df.shape)           # (50000, 13) — rows and columns
print(df.head())          # first five rows
print(df.info())          # dtypes and non-null counts — reveals
missing data
print(df["churn"].value_counts(normalize=True))   # class balance

# 2. Separate features from target
target = "churn"
drop_cols = ["customer_id", target]          # identifier and target
are not features
X = df.drop(columns=drop_cols)
y = df[target]

print(f"Features: {X.shape[1]} columns, {X.shape[0]} rows")
print(f"Churn rate: {y.mean():.1%}")
```

Line	Purpose
`df.shape`	Confirms you loaded what you expected — a first sanity check that catches wrong files

Line	Purpose
	and parsing errors
`df.info()`	Shows dtypes and non-null counts; a column typed `object` where you expected numbers signals a data problem
`value_counts(normalize=True)`	Reveals class imbalance immediately — the single most consequential property of a classification dataset
`df.drop(columns=drop_cols)`	Excludes the identifier (no signal, invites memorization) and the target (including it would be perfect leakage)
`y.mean()`	For a 0/1 column, the mean *is* the positive rate — a fast idiom worth knowing

Expected output (illustrative):

```
(50000, 13)
Features: 11 columns, 50000 rows
Churn rate: 4.2%
```

That 4.2% is a warning: a model predicting "no churn" for everyone achieves **95.8% accuracy** while being completely useless. This is exactly why Part 39's baselines exist and why accuracy is the wrong metric here.

---

26. Your First Machine Learning Model

A complete, explained, end-to-end example. Read every annotation — the code is the smallest part of the lesson.

26.1 Step 1 — Import Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report,
roc_auc_score
```

Import	Why it's needed
`pandas`	Loading and manipulating tabular data
`train_test_split`	Dividing data so evaluation is honest (Part 18)
`LogisticRegression`	A simple, fast, interpretable classifier — the right first choice
`StandardScaler`	Puts features on a comparable scale; logistic

Import	Why it's needed
	regression's optimizer converges better and coefficients become comparable
`Pipeline`	Chains preprocessing and model so that scaling is fit on training data only — **structurally preventing leakage**
Metrics	Quantifying performance — several, because one number is never enough

26.2 Step 2–3 — Load and Inspect

```
df = pd.read_csv("customer_churn.csv")

print(df.shape)
print(df.isnull().sum())                          # missing values per
column
print(df["churn"].value_counts(normalize=True)) # class balance
print(df.describe())                              # numeric summary
statistics
```

`df.describe()` deserves attention: check the `min` and `max` rows for impossible values (negative tenure, age of 200) and compare `mean` against `50%` (the median) — a large gap indicates skew and possible outliers.

26.3 Step 4–5 — Select Features and Target

```
# Use only numeric features for this first model, to keep it simple.
# Categorical encoding is covered properly in the preprocessing
module.
feature_cols = [
    "age",
    "tenure_months",
    "monthly_income",
    "monthly_spend",
    "num_transactions",
    "support_calls",
    "customer_satisfaction",
]

X = df[feature_cols]
y = df["churn"]
```

Why restrict to numeric here: `LogisticRegression` requires numeric input. `country` and `contract_type` need encoding first. Deliberately deferring that keeps this first model comprehensible — and honestly noting the simplification is better practice than silently dropping columns.

26.4 Step 6 — Split the Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
```

```
    test_size=0.2,          # hold out 20% for final evaluation
    random_state=42,        # fixes the shuffle so results are
reproducible
    stratify=y,             # preserve the churn rate in both splits
)

print(f"Training: {X_train.shape[0]} rows, churn rate
{y_train.mean():.1%}")
print(f"Test:     {X_test.shape[0]} rows, churn rate
{y_test.mean():.1%}")
```

Argument	Why it matters
`test_size=0.2`	Reserves genuinely unseen data for honest evaluation
`random_state=42`	**Reproducibility** — without it, every run gives different splits and different scores, making comparisons meaningless. The number itself is arbitrary.
`stratify=y`	**Essential with imbalance.** Without it, random chance could give the test set 3.1% churn and the training set 4.5%, distorting both training and evaluation

Expected output:

```
Training: 40000 rows, churn rate 4.2%
Test:     10000 rows, churn rate 4.2%
```

Matching rates confirm stratification worked.

26.5 Step 7–8 — Create and Train the Model

```
model = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000,
class_weight="balanced")),
])

model.fit(X_train, y_train)
```

Element	Explanation
`Pipeline([...])`	Bundles scaling and modelling into one object. When `.fit()` runs, the scaler learns means/standard deviations **from training data only**; when `.predict()` runs on test data, it *applies* those stored statistics. This is leakage prevention enforced by construction, not by discipline.
`StandardScaler()`	Transforms each feature to mean 0, standard

Element	Explanation
	deviation 1. Without it, `monthly_income` (values ~200,000) would numerically dominate `support_calls` (values ~3)
`max_iter=1000`	The optimizer's iteration ceiling. The default (100) often triggers a convergence warning on real data; raising it is the standard fix
`class_weight="balanced"`	**Important with 4% imbalance.** Tells the model to weight the rare class more heavily, so it doesn't simply learn "always predict no churn"
`.fit(X_train, y_train)`	Training: the optimizer searches for coefficients minimizing the loss (Part 19). This is the `argmin` from Section 19.2, executed

What happens internally (high level): logistic regression computes a weighted sum of features, passes it through a sigmoid function to produce a probability between 0 and 1, compares that to the true label using log loss, and iteratively adjusts weights to reduce that loss.

26.6 Step 9 — Make Predictions

```
y_pred  = model.predict(X_test)              # hard labels: 0 or 1
y_proba = model.predict_proba(X_test)[:, 1]  # probability of class 1 (churn)

print(y_pred[:10])
print(y_proba[:10].round(3))
```

Method	Returns	When to use
`.predict()`	Hard 0/1 decisions using a 0.5 threshold	When a binary decision is genuinely required
`.predict_proba()[:, 1]`	Probability of the positive class	**Usually preferable** — enables ranking and threshold tuning to the business's cost tradeoff

For the churn campaign (Section 25.3), `y_proba` is what matters: the team needs the top 2,000 by risk, which requires a *ranking*, not a binary flag.

26.7 Step 10 — Evaluate

```
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(f"ROC-AUC:  {roc_auc_score(y_test, y_proba):.3f}")
print(classification_report(y_test, y_pred, digits=3))
```

Expected output (illustrative):

```
Accuracy: 0.782
ROC-AUC:  0.831

              precision    recall  f1-score   support
           0      0.987     0.784     0.874      9580
           1      0.142     0.792     0.241       420
    accuracy                         0.783     10000
```

How to read this — and this interpretation is the real lesson of Part 26:

- **Accuracy 78.2%** is *lower* than the 95.8% a "predict nobody churns" model would achieve. That is not failure. Accuracy is simply the wrong metric under imbalance.
- **ROC-AUC 0.831** means: given one random churner and one random non-churner, the model ranks the churner as riskier 83% of the time. Since the campaign is a ranking exercise, **this is the number that matters**. 0.5 would be random guessing.
- **Recall for class 1 = 0.792** — the model catches 79% of actual churners. Good for the business objective.
- **Precision for class 1 = 0.142** — only 14% of flagged customers would genuinely have churned. Whether that is acceptable depends entirely on economics: if an offer costs ₦500 and a retained customer is worth ₦40,000, contacting seven people to save one is enormously profitable.

This precision/recall tradeoff — and its dependence on business costs rather than on any universally "good" number — is the single most important evaluation lesson in this module.

27. Machine Learning Project Structure

27.1 A Professional Layout

```
customer_churn_project/
│
├── data/
│   ├── raw/                    # Original, immutable data — NEVER edited
│   ├── interim/                # Partially processed
│   └── processed/              # Model-ready datasets
│
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_feature_exploration.ipynb
│   └── 03_model_experiments.ipynb
│
├── src/
│   ├── data/                   # Loading and validation code
│   ├── features/               # Feature engineering functions
│   ├── models/                 # Training and prediction code
│   └── evaluation/             # Metrics and reporting
│
├── tests/                      # Unit tests for src/
├── models/                     # Serialized trained models
(.pkl, .joblib)
├── reports/
│   └── figures/
│
├── requirements.txt            # Pinned dependencies
├── README.md                   # Problem, approach, how to run
├── .gitignore                  # Excludes data/ and models/ from Git
└── main.py                     # Entry point orchestrating the pipeline
```

27.2 Purpose of Each Component

Component	Purpose and rationale
`data/raw/`	The original data, **treated as read-only**. Every transformation is code applied to it, so any result can be regenerated from scratch. Editing raw data destroys reproducibility permanently.
`data/processed/`	Model-ready outputs, regenerable by re-running the pipeline
`notebooks/`	Exploration and communication. Numbered so the intended reading order is obvious. **Notebooks are for exploring, not for production** — their hidden execution order makes them unreliable as pipeline code
`src/`	The real, tested, importable code. Logic that

Component	Purpose and rationale
	matters migrates from notebook to `src/`
`tests/`	Unit tests — feature functions and data validation especially. ML code needs testing as much as any other software
`models/`	Serialized artefacts, versioned so a specific prediction can be traced to a specific model
`reports/`	Outputs for humans: metrics, figures, written findings
`requirements.txt`	**Pinned** versions. `scikit-learn==1.5.0`, not `scikit-learn` — an unpinned upgrade silently changing behaviour is a classic reproducibility failure
`README.md`	The business problem, the approach, how to run it, known limitations
`.gitignore`	Data and models don't belong in Git (size, and often confidentiality); use DVC or object storage with version tags

27.3 Reproducibility

Definition: anyone — including you in six months — can rerun the project and obtain the same results.

Requirement	How to achieve it
Same data	Immutable raw data, versioned (DVC, object storage with versioning)
Same code	Git
Same environment	Pinned `requirements.txt`, ideally a Docker image
Same randomness	`random_state` / seeds set everywhere randomness occurs
Same process	The pipeline is *code*, not a sequence of manually executed notebook cells

Reproducibility is not academic fastidiousness. In regulated industries — banking model risk management, medical devices — it is a **legal requirement**: you must be able to demonstrate exactly how a decision affecting a customer was produced.

---

28. Common Beginner Mistakes

#	Mistake	Why it's wrong	How to avoid it
1	**Thinking ML is**	Algorithm choice is a	Spend proportionally

#	Mistake	Why it's wrong	How to avoid it
	only about algorithms	small fraction of project outcome; data quality and problem framing dominate	more time on data and framing (Part 5)
2	**Skipping business understanding**	Produces a technically sound model solving the wrong problem	Define T, E and P and the business decision before writing code
3	**Using ML when it's unnecessary**	Adds cost, complexity and fragility over a working rule	Ask "would a simple rule achieve 90% of the value?" (Part 4)
4	**Training on the entire dataset**	Leaves no honest evaluation data	Always split first (Part 18)
5	**Testing on training data**	Measures memorization; scores are meaningless	Evaluate only on held-out data
6	**Data leakage**	Inflates scores; the model fails in production	Split first, transform after; scrutinize suspiciously good results
7	**Confusing features and targets**	Including the target (or a proxy) as a feature gives perfect, useless scores	Verify every feature is knowable *before* the prediction moment
8	**Using inappropriate metrics**	96% accuracy on 4% churn is worthless	Choose metrics matching the business objective (Part 26.7)
9	**Ignoring class imbalance**	The model learns to always predict the majority class	Stratify, use `class_weight`, evaluate with precision/recall/AUC
10	**Ignoring missing data**	Silently distorts results or crashes training	Quantify missingness; understand *why* it's missing; handle deliberately
11	**Ignoring outliers**	A few extreme values can dominate a model	Investigate them — they may be errors, or the most important cases (fraud!)

#	Mistake	Why it's wrong	How to avoid it
12	**Assuming complex = better**	Complex models overfit and are harder to maintain	Always start with a baseline (Part 39)
13	**Overfitting**	Excellent training scores, poor real performance	Monitor the train/test gap (Part 20)
14	**Underfitting**	Model too simple to capture the pattern	Diagnose with the Part 20.4 table
15	**Not validating models**	Hyperparameters tuned on test data invalidate the estimate	Use a validation set or cross-validation
16	**Randomly splitting time-series data**	Trains on the future, tests on the past	Time-based split, always, for temporal data
17	**Using future information**	A specific, severe form of leakage	Ask of every feature: "would I have this at prediction time?"
18	**Ignoring interpretability**	Unexplainable models are unusable in regulated settings and untrustworthy everywhere	Check feature importances; prefer simpler models where explanation is required
19	**Ignoring deployment**	A model in a notebook creates zero value	Plan deployment from the start (Part 27)
20	**Ignoring monitoring**	Models decay silently as the world changes	Monitor inputs, outputs and (as labels arrive) performance
21	**Focusing only on accuracy**	Hides catastrophic behaviour on the class you care about	Use a full classification report plus business-relevant metrics
22	**Not understanding the dataset**	Misinterpreting a column invalidates everything downstream	Talk to the people who generate the data
23	**Not setting random seeds**	Results can't be reproduced or compared	Set `random_state` everywhere
24	**Scaling before**	Test-set statistics	Use `Pipeline`

#	Mistake	Why it's wrong	How to avoid it
	splitting	leak into training	(Section 26.5)
25	**Deleting all rows with missing values**	Can discard most of the data, and biases the sample if missingness isn't random	Quantify impact before dropping; consider imputation

29. Machine Learning Business Use Cases

Format: Business problem → Data → Features → Target → ML type → Possible algorithm → Output → Business action.

1. Banking — Credit default prediction. Reduce loan losses → historical applications and outcomes → income, DTI, credit history, employment tenure, prior delinquencies → `defaulted` → binary classification → logistic regression or gradient boosting (explainability required) → default probability → approve/decline, set rate and limit.

2. Banking — AML transaction monitoring. Detect money laundering → transaction and counterparty data → amount patterns, velocity, counterparty risk, structuring indicators → `suspicious` (scarce labels) → classification + anomaly detection → isolation forest + gradient boosting → risk score → file a Suspicious Activity Report or clear.

3. FinTech — Real-time card fraud. Block fraudulent transactions in <100ms → transaction stream with confirmed fraud labels → amount, merchant category, geo-distance from last transaction, device, velocity → `is_fraud` (highly imbalanced) → binary classification → gradient boosting → fraud score → approve, challenge, or block.

4. Insurance — Claims severity prediction. Reserve capital accurately → historical claims → claim type, policy details, incident features, claimant history → `claim_amount` → regression → gradient boosting → expected payout → set reserves, prioritize adjuster assignment.

5. Insurance — Claims fraud detection. Reduce fraudulent payouts → claims and investigation outcomes → claim timing vs policy start, prior claims, provider patterns → `fraudulent` → classification → random forest → fraud likelihood → route to investigation.

6. Telecom — Subscriber churn. Retain revenue → CRM, billing, usage, support → tenure, spend trend, support calls, contract type → `churned_30d` → binary classification → gradient boosting → churn probability → targeted retention offers.

7. Telecom — Network fault prediction. Reduce outages → network element telemetry → error rates, utilization, temperature, historical faults → `fault_within_24h` → classification → gradient boosting → fault probability → dispatch preventive maintenance.

8. Oil & Gas — Production forecasting. Plan output and capital → well and reservoir history → pressure, choke size, water cut, well age, decline curve features → `barrels_per_day` → regression/time-series → gradient boosting or time-series models → forecast → production planning, investment decisions.

9. Oil & Gas — Predictive maintenance. Avoid unplanned downtime → equipment sensor telemetry + maintenance logs → vibration, temperature, pressure, rolling statistics → `failure_within_7d` → classification → gradient boosting → failure probability → schedule intervention.

10. Energy — Demand forecasting. Balance the grid → historical load → time of day, day type, temperature, holidays, historical load lags → `load_mw` → regression/time-series → gradient boosting → forecast → generation scheduling, purchasing.

11. Healthcare — Readmission risk. Reduce avoidable readmissions → EHR data → diagnoses, length of stay, comorbidities, medications, prior admissions → `readmitted_30d` → classification → logistic regression (interpretability essential) → risk score → enrol in post-discharge care programme.

12. Healthcare — Medical imaging triage. Speed up urgent diagnoses → labelled radiology images → learned pixel representations → `abnormality_present` → classification (deep learning, CNN) → abnormality probability → reprioritize the radiologist's worklist.

13. Retail — Demand forecasting per SKU/store. Optimize inventory → sales history → seasonality, price, promotions, weather, holidays → `units_sold` → regression/time-series → gradient boosting → forecast → purchase orders, allocation.

14. E-commerce — Product recommendation. Increase basket size → interaction history → user-item matrix, item attributes, session context → implicit engagement → recommendation/ranking → collaborative filtering or ranking models → ranked items → personalize the storefront.

15. E-commerce — Dynamic pricing. Maximize margin → price/demand history → competitor prices, inventory, demand elasticity features → `units_sold` at price → regression → gradient boosting → demand curve → set price.

16. Manufacturing — Visual defect detection. Reduce defective shipments → labelled product images → learned representations → `defective` → classification (CNN) → defect probability → divert item from the line.

17. Logistics — Delivery time estimation. Improve customer promises → historical deliveries → distance, route, traffic patterns, time of day, courier, weather → `delivery_minutes` → regression → gradient boosting → ETA → set customer expectations, plan routes.

18. Supply chain — Supplier risk scoring. Prevent disruption → supplier performance history → on-time rate, quality incidents, financial indicators, geography → `disruption_next_quarter` → classification → gradient boosting → risk score → diversify sourcing.

19. Marketing — Lead scoring. Focus sales effort → CRM and engagement data → firmographics, engagement events, source channel → `converted` → classification → logistic regression → conversion probability → prioritize the sales queue.

20. Marketing — Campaign response / uplift. Spend budget efficiently → past campaign outcomes → customer attributes, history, channel → `responded` → classification → gradient boosting → response probability → target the responsive.

21. Education — Student dropout risk. Improve completion rates → LMS and enrolment data → engagement, assignment submission, attendance, grades trend → `dropped_out` → classification → logistic regression → risk score → trigger academic support outreach.

22. Cybersecurity — Intrusion detection. Detect novel attacks → network traffic → packet statistics, connection patterns, protocol anomalies → often **unlabelled** → anomaly detection → isolation forest / autoencoder → anomaly score → alert the SOC.

23. Agriculture — Yield prediction. Plan harvest and pricing → field and weather history → soil metrics, rainfall, temperature, satellite vegetation indices → `yield_per_hectare` → regression → gradient boosting → forecast → planting and sales decisions.

24. Government — Tax fraud detection. Recover revenue → filings and audit outcomes → declared vs expected income patterns, sector benchmarks, prior audits → `fraudulent_filing` → classification → gradient boosting → risk score → select audit targets.

25. Transportation — Ride demand forecasting. Position supply efficiently → trip history → location, time, weather, events, historical demand → `rides_next_hour` → regression/time-series → gradient boosting → forecast → driver incentives and positioning.

26. Media — Content recommendation. Increase engagement → watch history → viewing patterns, content metadata, session context → implicit engagement → recommendation → collaborative filtering → ranked content → personalize the feed.

27. Real estate — Property valuation. Price listings accurately → historical sales → size, location, bedrooms, age, condition, neighbourhood indicators → `sale_price` → regression → gradient boosting → estimated value → pricing guidance, mortgage collateral assessment.

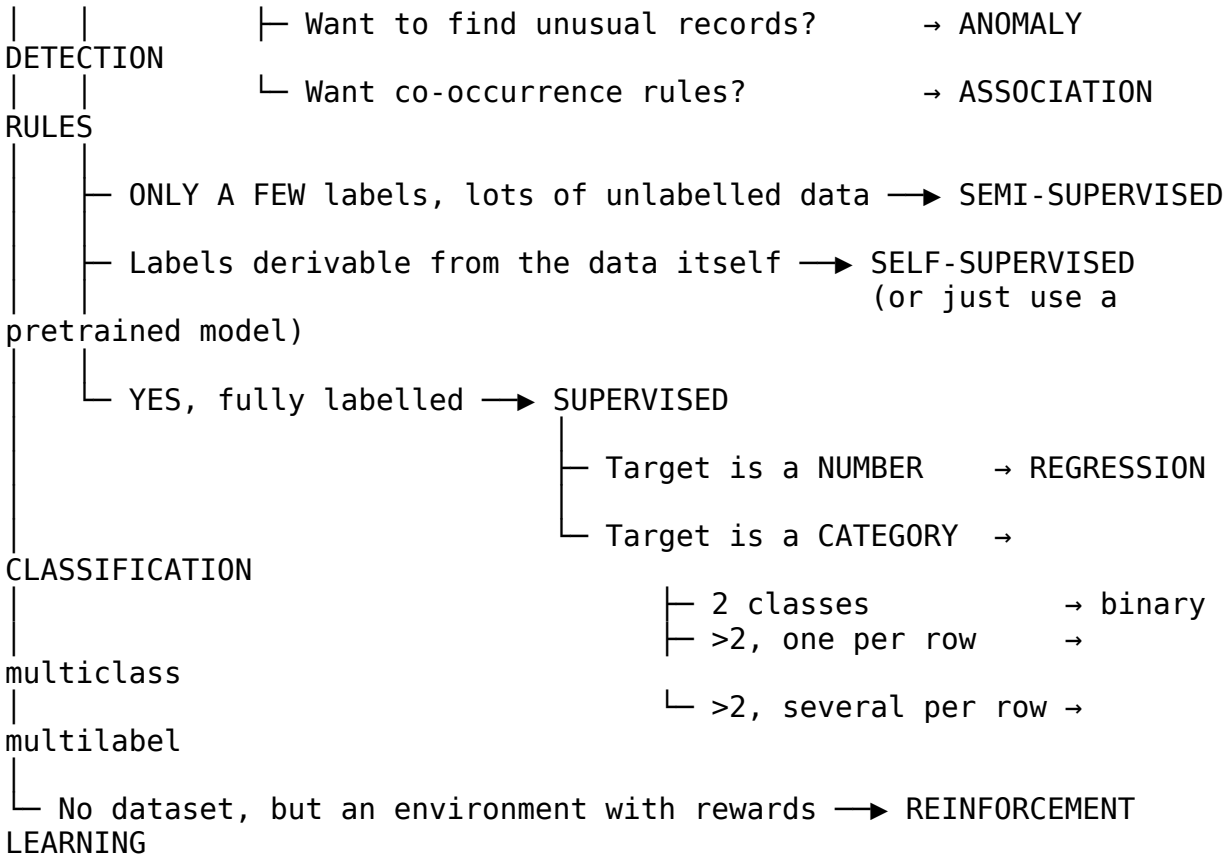
28. HR — Employee attrition. Retain key staff → HRIS data → tenure, compensation vs market, promotion history, engagement survey, manager changes → `left_within_12m` → classification → logistic regression (explainability and fairness critical) → attrition risk → retention conversations.

---

30. Machine Learning Decision Framework

30.1 The Core Decision Tree

```
START: What do you have and what do you want?
│
├─ Do you have a labelled target column?
│     │
│     ├─ NO ──► UNSUPERVISED
│     │           ├─ Want to group observations?          → CLUSTERING
│     │           ├─ Want to reduce features?            → DIMENSIONALITY
REDUCTION
```



30.2 Special Cases That Don't Fit Neatly

Situation	Treatment
Time series	Technically supervised regression, but **requires time-based splitting**, lag features, and awareness of trend/seasonality. Never randomly split it.
Ranking	The target may be binary (clicked/not) but the *objective* is ordering. Evaluate with ranking metrics (NDCG, precision@k), not accuracy.
Recommendation	Often implicit feedback (views, purchases) rather than explicit labels; specialized algorithms.
Anomaly detection	Can be unsupervised (learn "normal") or supervised (if labelled anomalies exist). With extreme imbalance, unsupervised often works better.
NLP	Text must be represented numerically first (embeddings). The task underneath is usually classification, generation or retrieval.
Computer vision	Images → classification, detection (where is

Situation	Treatment
	it?), or segmentation (which pixels?). Usually deep learning.

30.3 A Worked Application

"We have three years of equipment sensor readings and a maintenance log recording when each machine failed. We want to intervene before failures."

Walk the tree: Do we have a target? **Yes** — the maintenance log gives us failure events. Is it a number or a category? We could frame it either way: *"will it fail in the next 7 days?"* (category → **binary classification**) or *"how many days until failure?"* (number → **regression**). Is it temporal? **Yes** → time-based splitting mandatory, and features must be computed from windows *preceding* each prediction point.

Choosing between the two framings is a business decision: the classification framing is simpler, better suited to scarce failure events, and directly supports a "schedule maintenance" action. The regression framing gives richer information but is harder and more sensitive to noise. A professional would start with classification.

31. Machine Learning Tool Ecosystem

Orientation only — you will not learn these here, but you should recognize every name.

31.1 Python Core

Tool	What it's for
NumPy	Fast numerical arrays — the foundation everything else builds on
Pandas	Tabular data manipulation (DataFrames); the daily workhorse
Matplotlib	Base plotting library; maximum control, verbose
Seaborn	Statistical plots built on Matplotlib; concise for EDA
Plotly	Interactive charts; good for dashboards and exploration
SciPy	Scientific computing: statistics, optimization, signal processing

31.2 Classical Machine Learning

Tool	What it's for
Scikit-learn	The standard classical ML library: models, preprocessing, pipelines, metrics, model selection. **Learn this thoroughly first.**
Statsmodels	Statistical modelling with rigorous inference (p-values, confidence intervals) — where statistics matters more than prediction
XGBoost	Gradient boosting; long-time competition and industry standard
LightGBM	Gradient boosting optimized for speed and large datasets
CatBoost	Gradient boosting with excellent native categorical handling

31.3 Deep Learning

Tool	What it's for
TensorFlow	Google's deep learning framework; strong production tooling
Keras	High-level API (now part of TensorFlow); beginner-friendly
PyTorch	Meta's framework; dominant in research and increasingly in production

31.4 Data Ecosystem

Tool	What it's for
SQL	The single most important non-Python skill for a data professional
PostgreSQL / MySQL / SQL Server / Oracle	Operational relational databases — common data sources
BigQuery / Snowflake / Databricks	Cloud data warehouses and lakehouses where analytical data lives
Spark	Distributed processing for datasets exceeding one machine's memory

31.5 MLOps

Tool	What it's for
Git / GitHub	Version control — non-negotiable for professional work
MLflow	Experiment tracking, model registry, deployment
Weights & Biases	Experiment tracking and visualization
DVC	Version control for data and models
Docker	Containerization — reproducible environments
FastAPI	Serving models as HTTP APIs

31.6 Cloud

Provider	ML offering
AWS	SageMaker, Bedrock
Azure	Azure ML, Azure OpenAI
Google Cloud	Vertex AI

31.7 A Realistic Learning Order

Python → Pandas/NumPy → SQL → scikit-learn → visualization → gradient boosting → Git → one cloud platform → MLOps basics → deep learning → LLM/AI engineering. Attempting this list in parallel is the most common way beginners stall. Depth in scikit-learn and Pandas beats shallow familiarity with fifteen tools.

---

32. Machine Learning Career Map

Role	Focus	Core skills to build
Data Analyst	Describe what happened; support decisions	SQL, Excel, BI tools, statistics, communication
Data Scientist	Predict and prescribe;	Python, statistics, ML, SQL,

Role	Focus	Core skills to build
	experiment	experimental design, storytelling
ML Engineer	Run models reliably in production	Software engineering, Python, Docker, cloud, CI/CD, monitoring
AI Engineer	Build applications on foundation models	LLM APIs, prompting, RAG, vector databases, evaluation, system design
Data Engineer	Build the data foundation	SQL, Python, Spark, orchestration, warehouse architecture
ML Platform Engineer	Build the tooling other ML people use	Infrastructure, Kubernetes, platform engineering
MLOps Engineer	Automate the ML lifecycle	CI/CD, containers, monitoring, model registries
Research Scientist	Advance the methods themselves	Deep mathematics, publications, usually a PhD

32.1 What This Module Gives Every Path

Everyone above needs Module 1's content: understanding what ML can and cannot do, how problems are framed, why validation matters, and how models fail. An analyst who understands this collaborates far better with data scientists; a data engineer who understands it builds more useful pipelines; an AI engineer who understands it knows when a predictive model beats an LLM (Part 24).

Realistic sequencing: most people enter via analyst or engineer roles and move toward data science or ML engineering. Attempting to start directly as an ML engineer without software engineering fundamentals, or as a data scientist without SQL and statistics, is the most common source of frustration.

---

33. Practical Exercises

33.1 Beginner (1–12)

1. Define Machine Learning in your own words without using the word "learn."
2. State Mitchell's T, E and P for a spam filter.
3. Give three problems better solved by traditional programming than ML, with justification.
4. Classify: predicting tomorrow's temperature. Supervised or unsupervised? Regression or classification?
5. Classify: grouping customers by purchasing behaviour with no predefined groups.
6. Classify: deciding whether an email is spam.

7. Classify: predicting which of five product categories an item belongs to.
8. In a dataset with `customer_id, age, income, churn`, identify features, target, and any column that is neither.
9. Explain the difference between a parameter and a hyperparameter with one example of each.
10. Explain the difference between an algorithm and a model.
11. Why must we split data into training and test sets?
12. A model scores 99% on training data and 62% on test data. Diagnose it.

33.2 Intermediate (13–24)

13. A colleague randomly split three years of monthly sales data to forecast next month. Explain the flaw and the fix.
14. A fraud model reports 99.8% accuracy. The fraud rate is 0.2%. Explain why this may be worthless and what to measure instead.
15. You have 100,000 unlabelled X-rays and 300 labelled by a radiologist. Which paradigm applies, and why?
16. Explain why `StandardScaler` must be fit on training data only.
17. Give an example of target leakage in a loan default model.
18. A churn model includes `days_since_account_closed`. What is wrong?
19. Explain why `stratify=y` matters for a dataset with 3% positives.
20. Explain the difference between multiclass and multilabel classification with a business example of each.
21. When would you prefer `predict_proba()` over `predict()`?
22. Why is instance-based learning often unsuitable for low-latency production systems?
23. Explain the bias-variance tradeoff to a non-technical manager in four sentences.
24. Your model must explain individual decisions to a regulator. How does this constrain your algorithm choice?

33.3 Advanced Conceptual (25–34)

25. A model performs well in validation and poorly in production three months later, with no code change. List four possible causes.
26. You have 500 rows and 200 features. What is the primary risk, and what would you do?
27. Design the target variable definition for "customer churn" at a bank with current accounts. Consider at least three candidate definitions and their tradeoffs.
28. A retention campaign can contact 2,000 of 50,000 customers. Explain why ROC-AUC may be less useful than precision@2000 here.
29. Explain how self-supervised learning enabled modern LLMs, in a paragraph a beginner could follow.
30. When would you deliberately choose a worse-performing model over a better-performing one? Give three distinct scenarios.
31. Your predictive model and an LLM are both proposed for "identify high-risk transactions and explain the risk." How would you architect this?
32. A stakeholder asks for "95% accuracy." What questions do you ask before agreeing?

33. Describe how you would detect, at model-development time, that a feature will be unavailable at prediction time.
34. Explain why a model with excellent metrics might still be unfit to deploy.

33.4 Solutions

1. A method for building systems whose behaviour is derived from data rather than written by a programmer; the system adjusts internal parameters to improve a measured objective on data it has not seen.

2. T = classify emails as spam/not spam; E = a labelled archive of past emails; P = classification accuracy, and more importantly the false-positive rate on legitimate mail.

3. Tax computation (rules are published law, must be exact and auditable); payroll (deterministic and specified); interest accrual (a formula exists). ML would be less accurate, unexplainable and legally indefensible.

4. Supervised regression — we have historical temperatures (labels) and the target is numeric.

5. Unsupervised clustering — no predefined target exists.

6. Supervised binary classification.

7. Supervised multiclass classification (assuming an item belongs to exactly one category).

8. Features: `age`, `income`. Target: `churn`. Neither: `customer_id` — an identifier with no generalizable signal that invites memorization.

9. Parameter: learned from data (a regression coefficient). Hyperparameter: set before training (tree max depth, k in KNN).

10. The algorithm is the learning procedure ("random forest"); the model is the specific fitted result of running it on data.

11. To measure generalization rather than memorization — a sufficiently complex model can always fit training data perfectly, so training performance says nothing about future performance.

12. Overfitting: high training, low test, large gap. Fix by simplifying the model, adding data, or regularizing.

13. Random splitting lets the model train on future data and test on past data — impossible in production, so the estimate is fictitious and over-optimistic. Fix: time-based split (train on earlier periods, test on later).

14. Predicting "not fraud" for everything achieves 99.8%. Accuracy is dominated by the majority class. Measure precision, recall, F1 on the fraud class, ROC-AUC or PR-AUC, and ultimately the money saved versus investigation cost.

15. Semi-supervised learning — abundant unlabelled data plus scarce, expensive expert labels is precisely its defining situation. Alternatively, self-supervised pretraining on the unlabelled images followed by fine-tuning on the 300.

16. Fitting on the full dataset lets test-set means and standard deviations influence training — a form of leakage producing over-optimistic estimates. Use a `Pipeline` so scaling is fit inside each training fold.

17. Including `collections_department_contacted` as a feature — it only happens *after* default, so the model would achieve near-perfect training scores and be useless at application time.

18. The account is closed *because* the customer churned. The feature is a consequence of the target, unavailable at prediction time — target leakage.

19. Without stratification, random chance can produce splits with materially different positive rates (e.g. 1.8% vs 4.1%), distorting both training and the evaluation estimate; with only 3% positives the absolute counts are small enough for this to matter a lot.

20. Multiclass: each observation gets exactly one label (a product is Electronics *or* Apparel). Multilabel: several can apply at once (a support ticket tagged *billing* and *urgent* and *escalated*).

21. Whenever you need ranking (target the top N), threshold tuning to a business cost tradeoff, or a calibrated confidence — which is most of the time in business settings.

22. Prediction requires searching the stored training set, so serving cost grows with data size — the opposite of what low-latency, high-volume production demands.

23. "A model can be wrong in two ways. It can be too simple, so it misses the real pattern and is consistently off — that's bias. Or it can be too flexible, so it chases noise in our historical data and gives wildly different answers on new data — that's variance. Making the model more complex reduces one and increases the other. The job is finding the balance point where total error is lowest."

24. It pushes you toward inherently interpretable models (logistic regression, shallow trees, scorecards) or requires supplementing a complex model with explanation methods and documented validation. In many regulated contexts a slightly less accurate but explainable model is the correct professional choice.

25. (a) Data drift — the input distribution changed; (b) concept drift — the relationship between features and target changed; (c) a broken upstream data pipeline silently changing a feature's meaning or availability; (d) training-serving skew — features computed differently in production than in training.

26. Severe overfitting risk — with 200 features and 500 rows a flexible model can fit noise perfectly. Use a simple, heavily regularized model; reduce features via domain knowledge or selection; use cross-validation rather than a single split; and consider whether collecting more data is feasible before modelling at all.

27. Candidates: (i) explicit account closure — clean and unambiguous but misses dormant customers who are effectively gone; (ii) no transactions for 90 days — captures silent attrition but 90 is arbitrary and some customers are legitimately seasonal; (iii) balance below a threshold plus no activity — combines signals but adds complexity and more arbitrary

thresholds. Each produces a different label, class balance and business meaning; the choice must be made with the business, with an explicit prediction horizon attached.

28. ROC-AUC summarizes ranking quality across *all* thresholds, but the campaign only ever operates at one specific point — the top 4%. A model can have better overall AUC while being worse precisely in the region that will actually be used. Precision@2000 (and recall@2000) measure performance at the real operating point.

29. Language models are trained by hiding parts of text and asking the model to predict what was hidden. No human labels anything — the answer was already in the text. Doing this across billions of sentences forces the model to learn grammar, meaning, and factual associations, because all of that is needed to fill blanks well. The general-purpose internal representations acquired that way are what make the model useful for tasks it was never explicitly trained on.

30. (a) Regulatory explainability requirements make a less accurate but interpretable model mandatory; (b) latency limits — a slightly worse model that answers in 10ms beats a better one taking 2 seconds in a real-time payment path; (c) maintainability and risk — a simple model a team can actually monitor, debug and retrain beats a marginally better one nobody understands.

31. Use the predictive model for the *scoring and ranking* (calibrated numeric judgement over structured transaction data — LLMs do this poorly), and the LLM for *generating the written explanation* for the case file, grounded in the model's feature attributions and the retrieved transaction details. Never let the LLM produce the risk score itself.

32. Accuracy of what — on which class, at which threshold? What is the class balance (95% accuracy may be worse than doing nothing)? What decision does this drive, and what are the costs of false positives versus false negatives? What is the current baseline performance? Is 95% achievable given the data, or are we agreeing to an impossible target?

33. Trace each feature to its source system and ask *when* the value is written relative to the prediction moment. Check whether the field is populated at the time a new record is created versus updated later. A practical test: simulate scoring a record using only data available at its creation timestamp and see which features are null.

34. Because metrics are not the only criterion: it may be unexplainable in a context requiring explanation; it may exhibit unacceptable performance disparities across demographic groups (Part 40); it may be too slow for the latency budget; it may depend on features unavailable in production; the team may lack capacity to monitor and retrain it; or the business process to act on its output may not exist.

34. Mini Case Studies

Format: Problem → Data → Features → Target → ML type → Possible model → Evaluation concept → Business outcome.

1. Bank fraud detection. Card fraud losses rising → 18 months of transactions with confirmed fraud labels → amount, merchant category, geo-distance from previous transaction, time since last transaction, device fingerprint, velocity counts → `is_fraud` (0.15% positive) → binary classification, severely imbalanced → gradient boosting with class weighting → precision/recall at the review-capacity threshold; PR-AUC rather than accuracy; explicit cost model (a blocked legitimate transaction annoys a customer; a missed fraud costs money) → fewer losses with a controlled false-positive rate.

2. Telecom churn. 4% monthly churn → CRM, billing, usage, support records → tenure, spend trend, support-call rate, contract type, data usage change → `churned_30d` → binary classification → logistic regression baseline, then gradient boosting → precision@2000 (campaign capacity), lift over random targeting → retained revenue net of offer cost.

3. Credit default. Reduce loan losses while maintaining approval volume → historical applications with 24-month outcomes → income, DTI, credit history length, prior delinquencies, employment tenure → `defaulted_24m` → binary classification → regularized logistic regression (regulatory explainability) → AUC plus fairness metrics across protected groups; calibration matters because probabilities feed pricing → better risk-adjusted returns and a defensible model.

4. E-commerce recommendation. Flat basket size → 2 years of browsing and purchase events → user-item interactions, item attributes, session context → implicit engagement (click/purchase) → recommendation/ranking → collaborative filtering, then a learned ranker → precision@10, NDCG, and above all **online A/B test revenue per session** → increased basket size and engagement.

5. Oil production forecasting. Capital planning requires reliable output forecasts → well histories, reservoir data → pressure, water cut, well age, choke settings, decline-curve features → `barrels_per_day` (next 90 days) → regression / time-series → gradient boosting on engineered time features → MAE in barrels; crucially, **time-based validation** → better capital allocation and production planning.

6. Predictive maintenance. Unplanned downtime costs ₦12m per incident → 3 years of sensor telemetry plus maintenance logs → rolling means/std of vibration, temperature, pressure; operating hours; time since last service → `failure_within_7d` → binary classification, imbalanced → gradient boosting on windowed features → recall (missing a failure is expensive) balanced against unnecessary-maintenance cost; time-based split → reduced unplanned downtime.

7. Healthcare readmission risk. Readmission penalties and poor outcomes → EHR extracts → diagnoses, length of stay, comorbidity count, medication count, prior admissions, discharge disposition → `readmitted_30d` → binary classification → logistic regression (clinical interpretability essential) → recall at the care-programme capacity; fairness across demographics is mandatory → targeted post-discharge support, fewer readmissions.

8. Customer segmentation. Marketing treats all customers identically → transaction and profile data → recency, frequency, monetary value, category mix, channel preference → **no target** → unsupervised clustering → k-means with careful feature scaling → cluster stability, silhouette score, and above all business interpretability → differentiated campaigns per segment.

9. Marketing lead scoring. Sales wastes time on unqualified leads → CRM history of leads and outcomes → firmographics, engagement events, source channel, time-to-first-contact → `converted` → binary classification → logistic regression or gradient boosting → precision at the sales-capacity threshold; lift over current prioritization → higher conversion per rep hour.

10. Insurance claims severity. Reserves are set inaccurately → 5 years of closed claims → claim type, policy attributes, incident characteristics, claimant history, geography → `final_claim_amount` → regression → gradient boosting → MAE and calibration across severity bands (systematic under-reserving on large claims is far worse than random error) → accurate reserving and adjuster prioritization.

11. Cybersecurity anomaly detection. Novel attacks have no historical examples → network flow telemetry → packet statistics, connection patterns, protocol distributions, temporal features → **largely unlabelled** → anomaly detection (unsupervised) → isolation forest or autoencoder → alert volume versus analyst capacity; time-to-detection on known incidents → faster detection of previously unseen attack patterns.

12. Inventory forecasting. Stockouts and overstock simultaneously → 3 years of SKU-store sales → sales lags, seasonality, price, promotions, holidays, weather → `units_sold_next_week` → regression / time-series → gradient boosting with lag features → MAE weighted by item value; evaluated per SKU-store, time-split → reduced stockouts and working capital.

13. Real estate price prediction. Inconsistent listing prices → historical sales records → size, bedrooms, location, age, condition, neighbourhood indicators, distance to amenities → `sale_price` → regression → gradient boosting → MAE in currency and median absolute percentage error; note strong geographic clustering requires group-aware validation → consistent, defensible valuations.

14. Employee attrition. Losing key engineers → HRIS data → tenure, compensation vs market band, time since promotion, manager changes, engagement survey scores → `left_within_12m` → binary classification → logistic regression (explainability and fairness are critical; this affects people's careers) → recall at intervention capacity; **fairness audit mandatory** → proactive retention conversations.

15. Energy demand forecasting. Grid balancing and purchasing → years of load data → hour of day, day type, temperature, holidays, load lags → `load_mw` → regression / time-series → gradient boosting or specialized time-series models → MAPE at each forecast horizon; time-split, never random → better generation scheduling and lower purchase costs.

---

35. Interview Preparation

35.1 Beginner Questions (1–30)

1. What is Machine Learning? Building systems whose behaviour is learned from data rather than explicitly programmed, improving at a measured task with experience.

2. AI vs ML? AI is the broad field of building intelligent systems; ML is the subfield where behaviour is derived from data. All ML is AI; not all AI is ML (search, planning, expert systems are AI but not ML).

3. ML vs Deep Learning? DL is a subfield of ML using multi-layer neural networks that learn their own features. Classical ML relies on human-engineered features and generally wins on tabular data.

4. Supervised vs unsupervised? Supervised has labelled targets and learns a mapping; unsupervised has no labels and discovers structure.

5. Regression vs classification? Regression predicts a number; classification predicts a category.

6. Feature vs label? A feature is an input column; the label is the target column being predicted.

7. Training vs testing? Training data fits the model's parameters; test data measures performance on data the model has never seen.

8. Why split data? To measure generalization rather than memorization — a complex model can fit training data perfectly and still be useless.

9. What is overfitting? The model learns noise specific to the training data; training score high, test score low.

10. What is underfitting? The model is too simple to capture the real pattern; both scores poor.

11. What is generalization? Performing well on unseen data — the entire objective of ML.

12. Model vs algorithm? The algorithm is the learning procedure; the model is the fitted result.

13. Parameter vs hyperparameter? Parameters are learned from data during training; hyperparameters are set by you beforehand.

14. What is cross-validation? Splitting training data into k folds, training on k−1 and validating on the held-out fold, rotating, and averaging — a more stable and data-efficient estimate than a single split.

15. What is data leakage? When information unavailable at real prediction time influences training, producing performance estimates that cannot be reproduced in production.

16. What is inference? Using a trained model to produce predictions on new data.

17. Batch vs online learning? Batch trains on the full dataset periodically; online updates continuously as data arrives. Note this concerns *training*, not serving latency.

18. What is a loss function? A measure of how wrong a prediction is; training minimizes it.

19. What is bias (statistical)? Error from overly simple assumptions, causing underfitting.

20. What is variance? Error from sensitivity to the specific training sample, causing overfitting.

21. What is the bias-variance tradeoff? Increasing complexity reduces bias but increases variance; total error is minimized at an intermediate point.

22. Binary vs multiclass vs multilabel? Two classes; more than two but exactly one per observation; more than two with several applicable simultaneously.

23. What is a baseline model? A deliberately simple reference (majority class, mean, or existing business rule) that any real model must beat to justify its complexity.

24. What is class imbalance and why does it matter? When one class vastly outnumbers another; accuracy becomes misleading and models drift toward always predicting the majority class.

25. Why not use accuracy on imbalanced data? Predicting the majority class for everything scores extremely well while providing zero value.

26. What is precision? Of the cases predicted positive, the fraction actually positive.

27. What is recall? Of the actual positives, the fraction the model caught.

28. When do you prioritize recall over precision? When missing a positive is costly — disease screening, fraud, equipment failure. Prioritize precision when acting on a false positive is costly or capacity is scarce.

29. What is a decision threshold? The cutoff converting a predicted probability into a class decision; tuning it trades precision against recall.

30. What is deployment? Making the model available to the system that acts on its predictions — batch scoring or a real-time API.

35.2 Intermediate Questions (31–55)

31. Why must scaling be fit on training data only? Fitting on all data lets test statistics influence training — leakage that inflates estimates. Use `Pipeline` to enforce this structurally.

32. Why is stratified splitting important? With imbalance, random splits can produce materially different class rates across splits, distorting training and evaluation.

33. Why can you never randomly split time-series data? It trains on the future and tests on the past — impossible in production, producing fictitious scores.

34. What is target leakage? Give an example. A feature that is a consequence of the target: `account_closed_date` in a churn model, or `collections_contacted` in a default model.

35. Explain the difference between validation and test sets. Validation guides model selection and tuning (used repeatedly, becomes optimistically biased); test provides a single clean final estimate (used once).

36. Why is a single train/test split sometimes insufficient? On small data the estimate is high-variance — it depends heavily on which rows landed where. Cross-validation averages across folds.

37. What is ROC-AUC intuitively? The probability the model ranks a random positive above a random negative. 0.5 is random; 1.0 is perfect.

38. When is PR-AUC preferable to ROC-AUC? Under severe class imbalance, where ROC-AUC can look deceptively good because true negatives dominate.

39. What is model drift? Degradation over time as the world changes. *Data drift* = input distribution shifts; *concept drift* = the feature-target relationship itself changes.

40. How would you detect drift in production? Monitor input feature distributions, prediction distributions, and — as ground truth arrives — actual performance, alerting on significant deviation from training baselines.

41. Parametric vs non-parametric? Parametric assumes a functional form with a fixed parameter count; non-parametric makes fewer assumptions and grows in complexity with data.

42. Instance-based vs model-based? Instance-based stores examples and compares at prediction time (KNN); model-based fits a compact function and discards the data.

43. What is semi-supervised learning and when is it used? Learning from abundant unlabelled data plus scarce labels — used when labelling is expensive (medical imaging, fraud confirmation).

44. What is self-supervised learning? Generating labels automatically from the data's own structure (masked word prediction), used to learn general representations — the foundation of modern LLMs.

45. Why do gradient-boosted trees usually beat neural networks on tabular data? Tabular data has heterogeneous features, limited rows, and no spatial/sequential structure for deep architectures to exploit; boosted trees handle mixed types, non-linearities and interactions efficiently without needing massive data.

46. Explain the exploration-exploitation tradeoff. In RL (and recommendation), whether to take the currently-best-believed action or try an unfamiliar one that might be better. Too much exploitation locks in mediocrity; too much exploration never converges.

47. Why might you choose a less accurate model? Explainability requirements, latency constraints, maintainability, fairness, or reliance on features unavailable in production.

48. What is calibration and when does it matter? Whether predicted probabilities match observed frequencies (of cases predicted 0.7, about 70% should be positive). Critical when probabilities feed downstream decisions like pricing or expected-value calculations.

49. How do you handle class imbalance? Stratified splitting, class weighting, resampling, appropriate metrics (PR-AUC, recall), and threshold tuning to the business cost tradeoff — usually several of these together.

50. What's the difference between a batch-scoring and a real-time serving architecture? Batch computes predictions on a schedule and writes them to a table (simple, cheap, latency in hours); real-time serves an API per request (complex, requires feature availability at request time, latency in milliseconds).

51. What is training-serving skew? Features computed differently in training than in production, causing degradation invisible in offline evaluation.

52. Why is `random_state` important? Reproducibility — without it, every run gives different splits and scores, making comparisons between models meaningless.

53. What does `class_weight='balanced'` do? Weights each class inversely to its frequency in the loss function, so the rare class is not ignored.

54. When is clustering validation difficult, and how do you approach it? Always — there is no ground truth. Use internal metrics (silhouette), stability across resampling, and above all whether the clusters are interpretable and actionable to the business.

55. How do you decide the number of clusters? Elbow method, silhouette analysis, and business judgement. The statistical methods narrow the range; the business decides what's actionable.

35.3 Scenario Questions (56–70)

56. *"Your fraud model has 99.9% accuracy. The business is thrilled. What do you say?"* — Ask for the fraud rate. If it's 0.1%, a model predicting "never fraud" achieves exactly 99.9%. Request the confusion matrix, precision and recall on the fraud class, and the comparison against a majority-class baseline.

57. *"A model that worked for a year suddenly degrades. No code changed."* — Investigate drift first: has the input distribution shifted (new customer mix, new product), has the relationship changed (new fraud tactic, economic shift), or has an upstream pipeline silently altered a feature? Check monitoring dashboards before touching the model.

58. *"Stakeholders want you to add a feature that dramatically improves accuracy."* — Verify it's available at prediction time and is not a consequence of the target. A dramatic improvement from one feature is the classic leakage signature.

59. *"You have two weeks and no labelled data."* — Consider whether labels can be constructed from existing operational records; whether an unsupervised approach (anomaly detection, segmentation) answers the business question; whether a small manual labelling effort on a sample is feasible; or whether the honest answer is that this isn't an ML problem yet.

60. *"The model must explain every decision to a regulator."* — Constrain to inherently interpretable models (regularized logistic regression, shallow trees, scorecards), document the full development process, validate fairness across protected groups, and maintain reproducibility of every decision.

61. *"Your churn model is accurate but churn hasn't decreased."* — The model isn't the problem. Either nobody is acting on the scores, the intervention itself is ineffective, or you're targeting customers who would have stayed anyway (a *ranking* rather than *causation* problem — consider uplift modelling).

62. *"You have 50 features and 400 rows."* — High overfitting risk. Reduce dimensionality using domain knowledge, use a heavily regularized simple model, rely on cross-validation rather than a single split, and set expectations that results will be uncertain.

63. *"Predictions must return in under 50ms."* — This rules out instance-based methods at scale and very large ensembles; it also constrains feature engineering to what can be computed at request time. Design the feature pipeline around this from the start, not after modelling.

64. *"Marketing wants a model to predict who will respond to a campaign."* — Clarify whether they want *response prediction* (who will respond) or *uplift* (who will respond **because of** the campaign). These are different problems; targeting people who would have bought anyway wastes budget.

65. *"How would you approach a problem you've never seen before?"* — Start with the business decision and its value; define the target precisely with a horizon; audit data availability at prediction time; establish a baseline; frame the ML problem type (Part 30); then model — simplest first.

66. *"Your test score is much better than your validation score."* — Suspicious. Likely a small or unrepresentative test set, or an accidental difference in how the sets were constructed. Investigate before celebrating.

67. *"The business wants to use the model on a new country's customers."* — The training distribution may not cover that population. Validate on data from that country before deploying; expect degradation; consider whether retraining or a separate model is needed.

68. *"How do you know when a model is good enough to deploy?"* — When it beats the baseline by a margin that justifies its cost, meets the latency and explainability requirements, passes fairness checks, and the business process to act on it exists. Metrics alone never answer this.

69. *"An LLM could do this — why build a predictive model?"* — For calibrated numeric prediction over structured data, a trained model is more accurate, far cheaper per prediction, deterministic, explainable, and auditable. LLMs excel at language tasks, not at estimating probabilities from tabular features.

70. *"Explain your model to a non-technical executive."* — Lead with the decision it supports and its measured business impact, describe the top drivers in plain language, state the error rate in terms of concrete consequences, and be explicit about its limitations and when it should not be trusted.

36. Coding Challenges

Format: Question → Task → Hint → Solution → Explanation.

1. Create a NumPy array of 10 numbers and compute mean, median, standard deviation.

```
import numpy as np
a = np.array([12, 19, 31, 7, 25, 18, 22, 9, 14, 30])
print(a.mean(), np.median(a), a.std())
```

Explanation: `mean()` and `std()` are array methods; `median` is a NumPy function. Note `std()` defaults to population standard deviation (`ddof=0`); pass `ddof=1` for the sample version.

2. Load a CSV and print its shape, columns and dtypes.

```
import pandas as pd
df = pd.read_csv("customer_churn.csv")
print(df.shape); print(df.columns.tolist()); print(df.dtypes)
```

Explanation: Always the first three things to check. An unexpected `object` dtype on a numeric column signals parsing problems (stray text, currency symbols).

3. Count missing values per column and as a percentage.

```
missing = df.isnull().sum()
print(pd.DataFrame({"count": missing, "pct": (missing / len(df) *
100).round(2)}))
```

Explanation: Percentages matter more than counts — 500 missing out of 50,000 is minor; out of 600 is a crisis.

4. Show the class balance of a binary target.

```
print(df["churn"].value_counts())
print(df["churn"].value_counts(normalize=True).round(4))
```

Explanation: This single check determines your metrics, splitting strategy and class weighting.

5. Select numeric columns only.

```
numeric_df = df.select_dtypes(include="number")
print(numeric_df.columns.tolist())
```

6. Separate features and target, excluding the ID column.

```
X = df.drop(columns=["customer_id", "churn"])
y = df["churn"]
```

Hint: Dropping the identifier is not optional — it carries no generalizable signal and invites memorization.

7. Perform a stratified 80/20 split and verify the class rates match.

```
from sklearn.model_selection import train_test_split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          random_state=42, stratify=y)
print(y_tr.mean().round(4), y_te.mean().round(4))
```

8. Compute summary statistics and spot impossible values.

```
print(df.describe().T[["min", "max", "mean", "50%"]])
```

Explanation: Check `min`/`max` for impossibilities (negative tenure, age 200). A large gap between `mean` and `50%` indicates skew.

9. Create a derived ratio feature safely.

```
df["spend_per_month"] = df["monthly_spend"] /
df["tenure_months"].replace(0, np.nan)
```

Explanation: Replacing 0 with `NaN` avoids division-by-zero producing `inf`, which breaks most models.

10. One-hot encode a categorical column.

```
encoded = pd.get_dummies(df["contract_type"], prefix="contract",
drop_first=True)
print(encoded.head())
```

Explanation: `drop_first=True` avoids perfect collinearity among the dummy columns — important for linear models.

11. Train a logistic regression inside a leakage-safe pipeline.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([("scaler", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=1000,
class_weight="balanced"))])
model.fit(X_tr, y_tr)
```

12. Predict probabilities and inspect the top 10 highest-risk customers.

```
proba = model.predict_proba(X_te)[:, 1]
top = pd.Series(proba,
index=X_te.index).sort_values(ascending=False).head(10)
print(top.round(3))
```

13. Produce a full classification report.

```
from sklearn.metrics import classification_report
print(classification_report(y_te, model.predict(X_te), digits=3))
```

14. Compute ROC-AUC.

```
from sklearn.metrics import roc_auc_score
print(round(roc_auc_score(y_te, proba), 3))
```

Explanation: Uses probabilities, not hard labels — `roc_auc_score(y_te, y_pred)` is a common and misleading mistake.

15. Build a majority-class baseline and compare.

```
from sklearn.dummy import DummyClassifier
baseline = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)
print("Baseline accuracy:", round(baseline.score(X_te, y_te), 3))
print("Model accuracy:   ", round(model.score(X_te, y_te), 3))
```

Explanation: The point is to reveal that accuracy is uninformative here — the baseline may well "win."

16. Compute precision at the top 2,000 predictions (campaign capacity).

```
k = 2000
results = pd.DataFrame({"proba": proba, "actual": y_te.values})
topk = results.nlargest(k, "proba")
print(f"Precision@{k}: {topk['actual'].mean():.3f}")
print(f"Recall@{k}:    {topk['actual'].sum() /
results['actual'].sum():.3f}")
```

Explanation: This is the metric that actually matches the business constraint (Section 25.1).

17. Cross-validate instead of using a single split.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring="roc_auc")
print(scores.round(3), "mean:", scores.mean().round(3))
```

Explanation: Wide variation across folds signals high variance and an unstable model.

18. Inspect logistic regression coefficients for interpretability.

```
coefs = pd.Series(model.named_steps["clf"].coef_[0],
index=X_tr.columns)
print(coefs.sort_values(key=abs, ascending=False).round(3))
```

Explanation: Valid comparison across features **only because** they were scaled first. A large positive coefficient increases churn odds.

19. Perform a time-based split.

```
df_sorted = df.sort_values("snapshot_date")
cutoff = int(len(df_sorted) * 0.8)
train, test = df_sorted.iloc[:cutoff], df_sorted.iloc[cutoff:]
```

Explanation: The mandatory approach for temporal data (Part 18.4).

20. Save and reload a trained model.

```
import joblib
joblib.dump(model, "models/churn_model_v1.joblib")
loaded = joblib.load("models/churn_model_v1.joblib")
```

Explanation: Saving the whole `Pipeline` preserves the fitted scaler alongside the classifier — reloading just the classifier would silently break predictions.

21. Group-aware split so a customer never spans train and test.

```
from sklearn.model_selection import GroupShuffleSplit
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups=df["customer_id"]))
```

22. Check for duplicate rows before modelling.

```
print("Exact duplicates:", df.duplicated().sum())
print("Duplicate IDs:   ", df["customer_id"].duplicated().sum())
```

Explanation: Duplicates spanning train and test inflate scores.

---

37. Troubleshooting

#	Problem	Cause	Diagnosis	Fix	Prevention
1	`ValueError: could not convert string to float`	A categorical column passed to a numeric model	`df.dtypes`	Encode categoricals (one-hot/ordinal)	Check dtypes before fitting
2	`ValueError: Input contains NaN`	Missing values	`df.isnull().sum()`	Impute or drop deliberately	Handle missingness in the pipeline
3	Shape mismatch: `X has 7 features, expecting 11`	Train and predict used different columns	Compare column lists	Use the same `Pipeline` for both	Never rebuild feature lists manually
4	Model predicts only one class	Severe imbalance	`pd.Series(pred).value_counts()`	`class_weight='balanced'`, resampling, threshold tuning	Check class balance first
5	Accuracy suspiciously high (>99%)	Leakage or extreme imbalance	Check class balance; review each	Remove leaking features;	Audit features against the

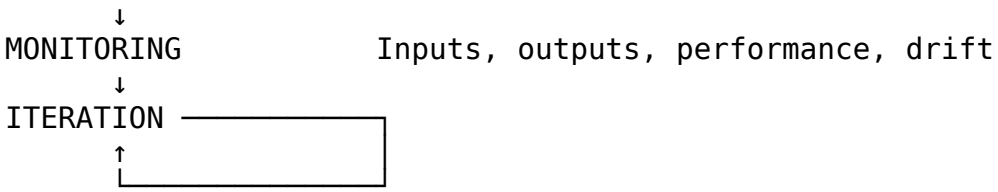
#	Problem	Cause	Diagnosis	Fix	Prevention
			feature's availability at prediction time	change metric	prediction timestamp
6	Test score far below training score	Overfitting	Compare train vs test explicitly	Simplify, regularize, add data	Always report both scores
7	Test score *above* training score	Usually a bug or tiny/unrepresentative test set	Inspect split sizes and construction	Re-examine splitting code	Use stratified splits and adequate test size
8	`ConvergenceWarning: lbfgs failed to converge`	Optimizer hit its iteration limit	Read the warning	`max_iter=1000`; scale features	Always scale before linear models
9	`y should be a 1d array`	Target passed as a DataFrame	Check `y.shape`	Use `df["churn"]`, not `df[["churn"]]`	Single brackets for the target
10	Unknown category at prediction time	A category absent from training data	Compare unique values	`handle_unknown='ignore'` in `OneHotEncoder`	Use encoders inside a pipeline
11	Results change on every run	No random seed	Rerun and compare	Set `random_state` everywhere	Seed as a standing habit
12	`MemoryError`	Dataset exceeds RAM	Check `df.memory_usage(deep=True).sum()`	Downcast dtypes, sample, chunk, or use Spark	Profile size early
13	Training takes hours	Too much data, too complex a model, or unscaled features	Time a small subsample first	Subsample for iteration; simpler model	Prototype on a sample
14	Notebook	Memory	Monitor	Delete	Avoid

#	Problem	Cause	Diagnosis	Fix	Prevention
	crashes	exhaustion, often from copies	memory	intermediates, restart kernel	unnecessary `.copy()`
15	`roc_auc_score` gives a strange low value	Hard labels passed instead of probabilities	Check the second argument	Pass `predict_proba(X)[:, 1]`	Remember: AUC needs scores
16	Precision is 0.0	Model never predicts the positive class	Inspect the confusion matrix	Address imbalance; lower the threshold	Check class balance first
17	Model great offline, poor in production	Training-serving skew or drift	Compare production feature distributions to training	Align feature computation; retrain	Monitor inputs continuously
18	Feature importances look nonsensical	Unscaled features, or leakage	Inspect the top features individually	Scale; remove leaking features	Review importances routinely — they catch leakage
19	Cross-validation scores vary wildly	High variance, small data	Inspect per-fold scores	Simpler model; more data	Always look at fold variance, not just the mean
20	`SettingWithCopyWarning`	Assigning to a slice of a DataFrame	Locate the chained indexing	Use `.loc` or an explicit `.copy()`	Avoid chained indexing
21	`inf` values break training	Division by zero in a derived feature	`np.isinf(X).sum()`	Replace zero denominators with `NaN` first	Guard every ratio feature
22	Dates treated as strings	No date parsing on load	`df.dtypes`	`pd.read_csv(..., parse_dates=["col"])`	Parse dates on load
23	Dataset too small for reliable results	Insufficient data	Check rows per class	Collect more; use cross-validation; simpler model; set expectations	Assess adequacy before modelling
24	Encoded	Encoders fit	Compare	Fit the	Always

#	Problem	Cause	Diagnosis	Fix	Prevention
	categories differ between train and test	separately	encoded column counts	encoder once inside a pipeline	pipeline preprocessing
25	Predictions are all identical	Model failed to learn (no signal, or a bug)	Inspect prediction variance	Verify features aren't constant; check the target isn't constant; try a stronger model	Sanity-check target and feature variance in EDA

38. Professional Machine Learning Workflow

```
PROBLEM                What business pain exists?
      ↓
BUSINESS OBJECTIVE     What decision will change, and what is that
worth?
      ↓
SUCCESS METRIC         Business metric first; model metric chosen to
match it
      ↓
DATA REQUIREMENTS      What features, at what grain, available when?
      ↓
DATA ACQUISITION       Pipelines, access, permissions, historical depth
      ↓
DATA VALIDATION        Schema, ranges, missingness, duplicates,
freshness
      ↓
EDA                    Distributions, relationships, class balance,
anomalies
      ↓
FEATURE ENGINEERING    Ratios, aggregates, time windows, encodings
      ↓
BASELINE               The simplest possible reference (Part 39)
      ↓
MODEL EXPERIMENTATION  Simple → complex, tracked and reproducible
      ↓
EVALUATION             On the untouched test set, with business-relevant
metrics
      ↓
INTERPRETATION         Why does it predict this? Does it make domain
sense?
      ↓
DEPLOYMENT             Batch or real-time; versioned artefact
```



38.1 Stage Notes That Separate Professionals From Beginners

Success metric before modelling. Agreeing the metric *after* seeing results invites choosing whichever metric flatters the model. Fix it in advance.

Data validation as a first-class step. Automated schema and range checks catch upstream breakage before it silently corrupts a model. This is the same discipline as testing in software engineering.

Baseline before experimentation. Without it you cannot tell whether your sophisticated model adds value.

Tracked experimentation. Record every run's data version, code version, hyperparameters and metrics (MLflow, W&B). "Which configuration produced last Tuesday's good result?" must be answerable.

Interpretation is not optional. Checking feature importances routinely catches leakage, catches nonsense, and builds the trust required for anyone to act on the output.

Iteration is the normal state. The first deployed model is the beginning, not the end.

39. Baseline Models

39.1 Why Baselines Matter

A **baseline** is the simplest possible reference, established *before* building anything sophisticated. Without one, a model's score is a number floating in space with no interpretation.

Three concrete reasons baselines are non-negotiable:

1. **They reveal whether the problem is even learnable.** If a strong model barely beats the baseline, the features likely lack signal — a finding worth discovering in week one, not week eight.
2. **They calibrate expectations.** 72% accuracy sounds mediocre until you learn the baseline is 51%; 96% sounds excellent until you learn the baseline is 95.8%.
3. **They justify complexity.** Every increment of model complexity carries a permanent cost in maintenance, latency and explainability. A complex model must *earn* that cost with a meaningful margin over the baseline.

39.2 Types of Baseline

Baseline	How it works	Appropriate for
Majority class	Always predict the most common class	Classification
Mean/median prediction	Always predict the training mean or median	Regression
Random	Predict randomly, respecting class proportions	Establishing a floor
Naive/persistence	Predict that tomorrow equals today (or last year's same period)	Time series — often surprisingly strong
Simple statistical	A single-feature rule ("tenure < 6 months → churn")	Any problem; often interpretable and decent
Business baseline	The current process, whatever it is — an existing rules engine, a manual heuristic, or human judgement	**The most important one**

39.3 The Business Baseline Is the One That Matters

The retention team may already target customers with more than three support calls. Your model's real competition is *that rule*, not a statistical abstraction. If your model achieves 0.83 AUC and the existing rule achieves 0.79, the honest framing is a modest incremental gain — which may or may not justify a production ML system with all its ongoing costs.

Teams that compare only against a random baseline systematically overstate their contribution.

39.4 In Code

```
from sklearn.dummy import DummyClassifier, DummyRegressor

# Classification baselines
DummyClassifier(strategy="most_frequent")   # always the majority class
DummyClassifier(strategy="stratified")      # random, respecting class proportions

# Regression baseline
DummyRegressor(strategy="mean")             # always the training mean
```

Reporting the baseline alongside every model result should be a standing habit — it is one line of extra output and it prevents an entire class of self-deception.

---

40. ML Ethics and Responsible Machine Learning

40.1 Why This Belongs in Module 1

Ethics is not an advanced topic to be appended after the technical material. The decisions that determine whether a system treats people fairly are made *early* — when choosing the target definition, selecting features, and sourcing training data. By the time a model is trained, many ethical properties are already locked in.

40.2 Bias and Fairness

Algorithmic bias means systematically different — and unjustified — outcomes across groups. Its sources are usually *data*, not malice:

Source	Example
Historical bias	Training a hiring model on past decisions that themselves discriminated teaches it to reproduce that discrimination
Representation bias	A medical model trained mostly on one demographic performs worse on others
Measurement bias	A proxy variable (arrest rate used to represent crime rate) carries the bias of the process that produced it
Aggregation bias	One model for all groups when the underlying relationships genuinely differ

Removing the sensitive attribute is not sufficient. Proxies persist — postcode correlates with race, purchasing patterns correlate with gender. A model can discriminate on a protected

attribute it never saw. Fairness requires *measuring outcomes across groups*, not merely omitting columns.

40.3 Privacy, Consent and Sensitive Attributes

Was the data collected with consent for *this* use? Does the model memorize and potentially expose individual records? Are special-category attributes (health, biometrics, religion, ethnicity) involved, triggering stricter legal requirements under GDPR, NDPR and similar regimes? Are you collecting more than you need — the data-minimization principle?

40.4 Explainability, Transparency, Accountability

Explainability: can you explain an individual decision? Legally required in credit (adverse action notices), and practically necessary anywhere a person is affected.

Transparency: do the people affected know a model is involved?

Accountability: who is responsible when it is wrong? "The model decided" is not an acceptable answer — a named human or team must own the outcome.

40.5 Domain Examples

Domain	The core risk
Banking / credit	Fair-lending violations via proxy discrimination; legally mandated explanations for declines
Healthcare	Models underperforming on underrepresented groups; over-trust in an automated triage that misses atypical presentations
Employment	Reproducing historical hiring discrimination at scale and speed
Insurance	Pricing that is technically predictive but socially or legally unacceptable
Criminal justice	Risk scores that encode enforcement bias and affect liberty
Education	Dropout-risk models becoming self-fulfilling if they alter how staff treat flagged students

40.6 Model Misuse and Automation Risk

A model built for one purpose gets repurposed for another it was never validated for. An automated system operates at a scale that makes its errors systematic rather than individual — one biased human affects the people they meet; one biased model affects everyone, consistently, immediately.

The practical countermeasures: a documented statement of intended use and known limitations; human review for high-stakes decisions; measured performance disaggregated

across groups; an appeals route for affected individuals; and monitoring that includes fairness metrics, not only accuracy.

Advanced ML governance, formal fairness metrics and regulatory frameworks come in a later module. What matters now is establishing the habit: **ask who could be harmed by this model, and how you would know.**

---

41. Final Comprehensive Project

41.1 Scenario

"Customer Churn Prediction — From Business Problem to First ML Model."

You are the first data scientist at a subscription telecom operator. Monthly churn is 4.2% of 50,000 customers. The retention team can contact 2,000 customers per month. A retention offer costs ₦500; an average retained customer is worth ₦40,000 in remaining lifetime value. The team currently targets anyone with more than three support calls in the last quarter.

41.2 Your Tasks

1. State the business problem, objective and success metrics (business and model).
2. Load and inspect the dataset.
3. Identify features, target, and columns that are neither.
4. Determine the ML problem type.
5. Explore the data — class balance, missingness, distributions.
6. Split the data correctly, justifying your strategy.
7. Build the **business baseline** (the existing >3 support calls rule) and a majority-class baseline.
8. Train a first model.
9. Generate probability predictions.
10. Evaluate at the real operating point (top 2,000).
11. Interpret the drivers.
12. Quantify the business implication in money.

41.3 Complete Solution

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report

# ---------- 2. LOAD AND INSPECT ----------
df = pd.read_csv("customer_churn.csv")
print(df.shape)
print(df.isnull().sum())
```

```
print(df["churn"].value_counts(normalize=True).round(4))

# ---------- 3-4. FEATURES, TARGET, PROBLEM TYPE ----------
# Target: churn (binary category) → SUPERVISED BINARY CLASSIFICATION,
imbalanced
feature_cols = ["age", "tenure_months", "monthly_income",
"monthly_spend",
                "num_transactions", "support_calls",
"customer_satisfaction"]
X = df[feature_cols]     # customer_id excluded: identifier, no
generalizable signal
y = df["churn"]          # gender excluded here: sensitive attribute,
see Part 40

# ---------- 6. SPLIT ----------
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# Note: with a snapshot_date column available, a TIME-BASED split
would be
# the professionally correct choice here (Part 18.4).

# ---------- 7. BASELINES ----------
# Business baseline: the existing rule
business_rule_pred = (X_te["support_calls"] > 3).astype(int)
business_auc = roc_auc_score(y_te, X_te["support_calls"])
print(f"Business rule AUC (support_calls alone): {business_auc:.3f}")

# Majority-class baseline accuracy
majority_acc = (y_te == 0).mean()
print(f"Majority-class accuracy: {majority_acc:.3f}")

# ---------- 8. TRAIN ----------
model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000,
class_weight="balanced")),
])
model.fit(X_tr, y_tr)

# ---------- 9. PREDICT ----------
proba = model.predict_proba(X_te)[:, 1]

# ---------- 10. EVALUATE AT THE OPERATING POINT ----------
print(f"Model ROC-AUC: {roc_auc_score(y_te, proba):.3f}")

k = int(len(y_te) * 0.04)          # campaign capacity, scaled to the
test set
res = pd.DataFrame({"proba": proba, "actual": y_te.values})
topk = res.nlargest(k, "proba")
```

```
precision_at_k = topk["actual"].mean()
recall_at_k    = topk["actual"].sum() / res["actual"].sum()
print(f"Precision@{k}: {precision_at_k:.3f}   Recall@{k}:
{recall_at_k:.3f}")

# ---------- 11. INTERPRET ----------
coefs = pd.Series(model.named_steps["clf"].coef_[0],
index=feature_cols)
print(coefs.sort_values(key=abs, ascending=False).round(3))

# ---------- 12. BUSINESS IMPLICATION ----------
contacted       = k
true_churners   = topk["actual"].sum()
offer_cost      = contacted * 500
# Assume the retention offer succeeds with 30% of genuine churners
contacted
saved_customers = true_churners * 0.30
revenue_saved   = saved_customers * 40_000
print(f"Contacted: {contacted} | True churners reached:
{true_churners}")
print(f"Cost: ₦{offer_cost:,.0f} | Est. revenue saved: ₦
{revenue_saved:,.0f}")
print(f"Estimated net: ₦{revenue_saved - offer_cost:,.0f}")
```

41.4 Expected Output and Interpretation

```
(50000, 13)
Churn rate: 4.2%

Business rule AUC (support_calls alone): 0.712
Majority-class accuracy: 0.958

Model ROC-AUC: 0.831
Precision@400: 0.318   Recall@400: 0.302

tenure_months           -0.742
support_calls            0.611
customer_satisfaction   -0.508
monthly_spend           -0.244
num_transactions        -0.187
monthly_income          -0.093
age                      0.041

Contacted: 400 | True churners reached: 127
Cost: ₦200,000 | Est. revenue saved: ₦1,524,000
Estimated net: ₦1,324,000
```

Reading this like a professional:

- **The model (0.831 AUC) beats the business rule (0.712) meaningfully** — this is the comparison that justifies the project. Comparing only against random would have overstated the contribution.
- **Precision@400 = 0.318** means roughly 1 in 3 contacted customers was a genuine churner, versus 4.2% by random targeting — **a 7.6× lift**. That lift, not accuracy, is the model's value.
- **Coefficient signs make domain sense:** longer tenure and higher satisfaction reduce churn risk (negative); more support calls increase it (positive). If a sign contradicted domain knowledge, that would demand investigation before deployment — often a leakage or data-quality signal.
- **The economics are decisively positive** even under conservative assumptions, and the calculation makes explicit the assumption (30% save rate) that most determines the answer — which should itself be measured, not assumed, once the campaign runs.

41.5 Extension Challenges

1. Add `country`, `contract_type` and `payment_method` via one-hot encoding. Does AUC improve?
2. Replace logistic regression with gradient boosting. Does the gain justify the loss of coefficient interpretability?
3. Implement a proper time-based split. How much does performance drop, and why is the lower number the more honest one?
4. Vary the assumed save rate from 10% to 50%. At what rate does the campaign stop being profitable?
5. Measure precision and recall separately for each `country`. Are outcomes equitable across groups (Part 40)?
6. Build a feature for *spend trend* rather than spend level. Does a declining trend predict churn better than the absolute value?

42. Final Knowledge Check

42.1 Multiple Choice (1–30)

1. ML is best described as: (a) writing rules by hand (b) **deriving behaviour from data** (c) storing data (d) faster computing
2. Which is NOT supervised? (a) regression (b) classification (c) **clustering** (d) binary classification
3. Predicting house price is: (a) classification (b) **regression** (c) clustering (d) RL
4. Predicting spam/not spam is: (a) regression (b) **binary classification** (c) clustering (d) forecasting
5. Customer segmentation with no predefined groups is: (a) supervised (b) **unsupervised** (c) semi-supervised (d) RL
6. A hyperparameter is: (a) learned from data (b) **set before training** (c) the output (d) a feature
7. Overfitting shows: (a) low train, low test (b) **high train, low test** (c) high both (d) low train, high test
8. Underfitting shows: (a) **low train, low test** (b) high train, low test (c) high both (d) none
9. `stratify=y` in a split: (a) shuffles (b) **preserves class proportions** (c) scales (d) removes outliers
10. Data leakage causes: (a) slower training (b) **over-optimistic scores** (c) missing values (d) imbalance
11. Time-series data must be split: (a) randomly (b) **by time** (c) stratified (d) never
12. `predict_proba()` returns: (a) hard labels (b) **probabilities** (c) accuracy (d) coefficients
13. With 1% positives, accuracy is: (a) the best metric (b) **misleading** (c) undefined (d) always low
14. Recall measures: (a) of predicted positives, how many are right (b) **of actual positives, how many were caught** (c) overall correctness (d) ranking
15. Precision measures: (a) **of predicted positives, how many are right** (b) of actual positives, how many were caught (c) overall correctness (d) error
16. A baseline exists to: (a) improve accuracy (b) **provide a reference for judging value** (c) clean data (d) deploy
17. Self-supervised learning: (a) needs human labels (b) **generates labels from the data itself** (c) uses rewards (d) is unsupervised clustering
18. RL learns from: (a) labels (b) **rewards** (c) clusters (d) embeddings
19. Deep learning most outperforms classical ML on: (a) small tables (b) **images and text** (c) regression (d) all data
20. For tabular business data, the usual strongest choice is: (a) CNN (b) KNN (c) **gradient-boosted trees** (d) k-means
21. Batch vs online learning refers to: (a) serving latency (b) **training frequency** (c) data size (d) hardware
22. KNN is: (a) parametric (b) **instance-based** (c) deep learning (d) unsupervised

23. High bias corresponds to: (a) overfitting (b) **underfitting** (c) leakage (d) imbalance
24. High variance corresponds to: (a) **overfitting** (b) underfitting (c) good fit (d) bias
25. The test set should be used: (a) during tuning (b) every epoch (c) **once, at the end** (d) for training
26. `random_state` ensures: (a) better accuracy (b) **reproducibility** (c) faster training (d) balance
27. Including `customer_id` as a feature: (a) improves accuracy (b) **adds no generalizable signal and risks memorization** (c) is required (d) prevents leakage
28. A column recorded after the outcome you predict is: (a) a good feature (b) **leakage** (c) the target (d) an identifier
29. Generative AI differs from predictive ML by: (a) using more data (b) **producing new content rather than labels/numbers** (c) being faster (d) needing no training
30. Removing a sensitive attribute from features: (a) guarantees fairness (b) **does not guarantee fairness because proxies remain** (c) is illegal (d) improves accuracy

42.2 Short Answer (31–50)

31. Define generalization. → Performing well on unseen data.
32. State Mitchell's definition components. → Task T, Experience E, Performance measure P.
33. Why can't training performance be trusted? → A complex model can memorize training data perfectly regardless of whether it learned anything transferable.
34. Difference between validation and test sets? → Validation guides tuning (used repeatedly, becomes biased); test gives one clean final estimate.
35. What is class imbalance? → One class vastly outnumbering another, making accuracy misleading and biasing models toward the majority.
36. Name three splitting strategies and when each applies. → Random (independent data); stratified (imbalanced classification); time-based (temporal data). Also group-based (repeated entities).
37. What is a parameter? → A value learned from data during training.
38. What is a loss function? → A measure of prediction error that training minimizes.
39. Explain `argmin` in $\theta^* = \text{argmin } L(\theta)$. → The parameter values that make the loss smallest.
40. Why square errors? → To make them positive (so they don't cancel) and to penalize large errors disproportionately.
41. What is model drift? → Performance degradation over time as data or relationships change.
42. Data drift vs concept drift? → Input distribution changes vs the feature-target relationship itself changes.
43. What is training-serving skew? → Features computed differently in production than in training.
44. Name two reasons to prefer a simpler model. → Explainability requirements; maintainability/latency.
45. What does `class_weight='balanced'` do? → Weights classes inversely to frequency so the rare class isn't ignored.

46. Why use a Pipeline? → It fits preprocessing on training data only, structurally preventing leakage, and keeps train/predict consistent.
47. What is ROC-AUC? → The probability the model ranks a random positive above a random negative.
48. When is PR-AUC better than ROC-AUC? → Under severe class imbalance.
49. What is a business baseline? → The current process (rule or human judgement) the model must beat to justify itself.
50. Why is ethics a Module 1 topic? → The decisions determining fairness are made early — target definition, feature selection, data sourcing — before any model exists.

42.3 Scenario Questions (51–60)

51. *99.9% accuracy on 0.1% fraud.* → Meaningless; equals the majority-class baseline. Demand precision/recall on the fraud class.
52. *Model degrades after a year, no code change.* → Drift (data or concept), or upstream pipeline change. Check monitoring first.
53. *One new feature causes a dramatic accuracy jump.* → Suspect leakage; verify availability at prediction time.
54. *100,000 unlabelled images, 300 labelled.* → Semi-supervised, or self-supervised pretraining then fine-tuning.
55. *Regulator requires per-decision explanations.* → Use inherently interpretable models; document validation; audit fairness.
56. *Accurate churn model, churn unchanged.* → Nobody acted, the intervention is ineffective, or you targeted people who'd have stayed anyway (needs uplift modelling).
57. *400 rows, 50 features.* → Severe overfitting risk; reduce features, regularize heavily, use cross-validation, set expectations.
58. *Predictions needed in under 50ms.* → Rules out instance-based methods at scale; constrains features to what's computable at request time.
59. *Test score higher than validation score.* → Suspicious; likely a small or unrepresentative test set, or a split bug.
60. *Deploying a model to a new country.* → Training distribution may not cover that population; validate on local data before deploying.

42.4 Practical Questions (61–70)

61. Write the code to check class balance. → `df["target"].value_counts(normalize=True)`
62. Write a stratified 80/20 split. → `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`
63. How do you verify stratification worked? → Compare `y_train.mean()` and `y_test.mean()`.
64. Write a pipeline with scaling and logistic regression. → See Section 26.5.
65. How do you get churn probabilities? → `model.predict_proba(X_test)[:, 1]`
66. How do you compute precision at the top 500? → Sort by probability, take the top 500, compute the mean of the actual labels.

67. How do you build a majority-class baseline? → `DummyClassifier(strategy="most_frequent")`
68. How do you save a fitted pipeline? → `joblib.dump(model, "model.joblib")` — save the whole pipeline, not just the classifier.
69. How do you perform a time-based split? → Sort by date, take the first 80% as train and the remainder as test.
70. How do you inspect logistic regression drivers? → Read `.coef_` as a Series indexed by feature names — valid only if features were scaled.

42.5 Mini Case Studies (71–75)

71. A hospital wants to flag patients at risk of readmission. Identify: ML type (supervised binary classification); target (`readmitted_30d`, with an explicit horizon); a key constraint (interpretability for clinicians, plus fairness across demographics); the metric (recall at care-programme capacity); one leakage risk (any post-discharge field recorded after the readmission window opens).

72. A bank's fraud rate is 0.1% and the review team handles 200 cases/day. Identify: the correct metric (precision@200 and recall@200, not accuracy); the splitting strategy (time-based); the imbalance handling (class weighting plus threshold tuning); and why unsupervised anomaly detection might complement the supervised model (novel fraud patterns have no labels).

73. A retailer has 3 years of daily sales and wants next week's demand per SKU. Identify: ML type (supervised regression / time-series); the mandatory split (time-based); an essential feature family (lags and seasonality); a strong baseline (last week's sales — persistence); and the evaluation (MAE weighted by item value).

74. An HR team wants to predict which employees will resign. Identify: ML type (supervised binary classification); two ethical requirements (fairness audit across protected groups; transparency about the system's existence and use); why explainability matters (decisions affect careers); and a major leakage risk (resignation-notice-related fields).

75. A cybersecurity team wants to detect novel intrusions with no labelled attack data. Identify: ML type (unsupervised anomaly detection); why supervised learning fails here (novel attacks have no historical examples by definition); the operational constraint (alert volume must match analyst capacity); and the evaluation approach (time-to-detection on known past incidents, plus false-positive rate).

---

43. Module Summary

Concept	The essential takeaway
AI	Building systems that perform tasks requiring intelligence; ML is a subfield, DL a subfield of that, generative AI/LLMs built on DL. Domains (CV, NLP) and paradigms (supervised, RL) are *orthogonal axes*, not one

Concept	The essential takeaway
	hierarchy
ML	Behaviour derived from data rather than written as rules; Mitchell's T/E/P makes it precise
Why ML exists	An enormous class of valuable problems has rules nobody can state
Traditional vs ML	Rules in → answers out, versus answers in → rules out
Lifecycle	Business problem → data → cleaning → EDA → features → training → validation → evaluation → deployment → monitoring → retraining. `fit()`/`predict()` is ~5% of the work
Features	Inputs; their quality caps model performance more than algorithm choice does
Target	The output; its presence and type determine the entire problem framing
Supervised	Labelled data; regression (numbers) or classification (categories); the enterprise workhorse
Unsupervised	No labels; clustering, dimensionality reduction, anomaly detection; validation is hard
Semi-supervised	Few labels, much unlabelled data; used where labelling is expensive
Self-supervised	Labels generated from the data itself; the foundation of LLMs and modern AI
Reinforcement	Learning from rewards through interaction; needs a safe environment to explore
Regression/Classification	Predicting numbers vs categories; binary, multiclass, multilabel
Train/Validation/Test	Training fits; validation tunes; test is touched once. Split *first*, transform after
Data leakage	Information unavailable at prediction time inflating scores; the most damaging common error
Learning mathematically	$\theta^* = \text{argmin } L(\theta)$ — find the parameters minimizing error
Generalization	Performing on unseen data; the entire objective
Overfitting/Underfitting	Learned noise vs too simple; diagnosed by the

Concept	The essential takeaway
	train/test gap
Bias/Variance	Wrong assumptions vs sensitivity to the sample; complexity trades one for the other
Baselines	The reference that makes any score interpretable; the business baseline matters most
Workflow	Professional practice spans problem framing through monitoring, with tracked experiments
Responsible ML	Fairness, privacy, explainability and accountability are decided early, not appended late

Where you now stand: you can identify what kind of ML problem a business situation represents, name its features and target, choose an appropriate splitting strategy, recognize leakage and imbalance, establish a baseline, train and evaluate a first model, read its output critically, and articulate why a model might be unfit to deploy despite good metrics. That foundation makes the upcoming modules — preprocessing, regression, classification, trees, ensembles, evaluation and feature engineering — substantially easier, because you now understand what those techniques are *for*.

---

44. Master Cheat Sheet

44.1 Definitions at a Glance

Term	One line
Machine Learning	Learning behaviour from data instead of explicit rules
Feature (X)	An input column
Target (y)	The column being predicted
Model	The fitted result of running an algorithm on data
Parameter	Learned from data
Hyperparameter	Set before training
Training	Finding parameters that minimize loss
Inference	Using a trained model on new data
Generalization	Performing well on unseen data
Overfitting	High train score, low test score
Underfitting	Low train score, low test score
Leakage	Using information unavailable at prediction

Term	One line
	time
Baseline	The simple reference a real model must beat

44.2 The ML Types

```
Labels?  NO → Unsupervised (clustering / dim. reduction / anomaly /
association)
         FEW → Semi-supervised
         AUTO-GENERATED → Self-supervised
         YES → Supervised → numeric target?  YES → Regression
                                             NO  → Classification
                                                   (binary /
multiclass / multilabel)
Environment + rewards → Reinforcement Learning
```

44.3 The Lifecycle

```
Business problem → Data → Understand → Clean → EDA → Features → Target
→ Split → Algorithm → Train → Validate → Evaluate → Tune → Interpret
→ Deploy → Monitor → Retrain (loop)
```

44.4 Key Formulas and Ideas

Idea	Expression
Model prediction	$\hat{y} = f_\theta(X)$
Training objective	$\theta^* = \text{argmin}_\theta L(\theta)$
Squared error	$(\hat{y} - y)^2$
Precision	TP / (TP + FP) — of those flagged, how many were right
Recall	TP / (TP + FN) — of actual positives, how many were caught
Majority-class accuracy	The fraction of the largest class — your minimum bar

44.5 Critical Distinctions

A	B
Algorithm (procedure)	Model (fitted result)
Parameter (learned)	Hyperparameter (set by you)
Validation (tune, repeatedly)	Test (evaluate, once)
Regression (number)	Classification (category)
Multiclass (one label each)	Multilabel (several labels each)
Batch (training frequency)	Online (training frequency) — *not* serving latency
Instance-based (store examples)	Model-based (fit a function)
Parametric (fixed parameter count)	Non-parametric (grows with data)

A	B
Bias (underfitting)	Variance (overfitting)
Predictive ML (labels/numbers)	Generative AI (new content)

44.6 The scikit-learn Workflow

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, classification_report
from sklearn.dummy import DummyClassifier

df = pd.read_csv("data.csv")
X, y = df.drop(columns=["id", "target"]), df["target"]

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

baseline = DummyClassifier(strategy="most_frequent").fit(X_tr, y_tr)

model = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000,
class_weight="balanced")),
]).fit(X_tr, y_tr)

proba = model.predict_proba(X_te)[:, 1]
print("Baseline:", baseline.score(X_te, y_te))
print("AUC:", roc_auc_score(y_te, proba))
print(classification_report(y_te, model.predict(X_te), digits=3))
```

44.7 Red Flags Checklist

- ☐ Accuracy above 99% → check imbalance and leakage
- ☐ One feature dominates importances → check leakage
- ☐ Random split on temporal data → invalid results
- ☐ Scaling before splitting → leakage
- ☐ No baseline reported → the score is uninterpretable
- ☐ `random_state` not set → not reproducible
- ☐ Test set used more than once → estimate is contaminated
- ☐ Identifier column used as a feature → memorization risk
- ☐ Model excellent offline, poor in production → skew or drift
- ☐ Fairness never measured across groups → unassessed ethical risk

44.8 Interview Concept Map

Always be ready to explain: ML vs AI vs DL; supervised vs unsupervised; regression vs classification; parameter vs hyperparameter; algorithm vs model; overfitting vs underfitting; bias vs variance; training vs validation vs test; data leakage with an example; why accuracy fails under imbalance; precision vs recall and when each dominates; what a baseline is and why it matters; batch vs online; instance-based vs model-based; parametric vs non-parametric; what drift is and how you'd detect it; and why an excellent model might still be unfit to deploy.

Conclusion

This module deliberately spent far more time on framing, data, validation and evaluation than on algorithms — because that is where real machine learning projects succeed or fail. A practitioner who can call `fit()` but cannot tell a leaking feature from a legitimate one, or a meaningful score from a meaningless one, will produce confident, useless models. A practitioner who has internalized this module's content will ask the right questions before writing any code, and will recognize a suspicious result before a stakeholder does.

The next modules go deep into technique: data preprocessing, regression, classification, decision trees, ensembles, model evaluation and feature engineering. Each will make more sense because you now understand what those techniques are *for* — they are all machinery in service of the single objective introduced here: **building a model that performs well on data it has never seen, in support of a decision that matters.**

End of Module 1

Kajola Gbenga | Data Scientist & AI Engineer